

- ☐ No syntax highlighting
- ☐ Hide deleted text

# Completing the Mathematical Model for C++ Quantities and Units

Document #: P4185R0 [[Latest](#)] [[Status](#)]  
Date: 2026-05-11  
Project: Programming Language C++  
Audience: SG6 Numerics  
Reply-to: Mateusz Pusz  
<[mateusz.pusz@gmail.com](mailto:mateusz.pusz@gmail.com)>

## Contents

- [1 Abstract](#)
- [2 Introduction](#)
  - [2.1 Design philosophy](#)
- [3 Motivation and scope](#)
  - [3.1 The temperature trap](#)
  - [3.2 Non-negative quantities lack a mathematical home](#)
    - [3.2.1 Preconditions as a symptom](#)
    - [3.2.2 Why the structure matters more than the checks](#)
  - [3.3 Mass balance and accumulation](#)
  - [3.4 Limitations of `relative\_point\_origin`](#)
    - [3.4.1 Structural type requirements](#)
    - [3.4.2 Parameter-dependent and runtime-determined origins](#)
    - [3.4.3 Scale, translation, and rotation](#)
  - [3.5 Bounded domains](#)
  - [3.6 Points without output](#)
  - [3.7 Affine relationships in quantity hierarchies](#)
  - [3.8 Integer division hazard](#)
  - [3.9 Comparison against zero](#)
- [4 Non-negative quantities](#)
  - [4.1 Overview](#)
  - [4.2 The `non\_negative` tag](#)

- 4.3 Inheritance through the hierarchy
- 4.4 Why automatic inference cannot be implemented correctly
  - 4.4.1 Inference from named spec definitions
  - 4.4.2 Inference at call-site arithmetic
- 4.5 Runtime enforcement on quantity point
- 4.6 Why a compile-time tag is not enough on its own
- 5 Absolute quantities
  - 5.1 Overview
  - 5.2 Syntax
  - 5.3 Absolute quantities are not points
  - 5.4 Scalar and vector quantities
  - 5.5 Conversions between abstractions
  - 5.6 Arithmetic semantics
    - 5.6.1 Addition
    - 5.6.2 Subtraction
    - 5.6.3 Multiplication and division
    - 5.6.4 Runtime enforcement of the `non_negative` flag
    - 5.6.5 Why checks cannot be further elided
    - 5.6.6 Unary negation
    - 5.6.7 Multiplication by a negative scalar
  - 5.7 Design rationale: the zero-anchor principle
    - 5.7.1 The zero-anchor principle
    - 5.7.2 Consistency with `Absolute × Scalar → Absolute`
    - 5.7.3 Handling underflow: when `delta > absolute`
    - 5.7.4 Algebraic-equivalence asymmetry
  - 5.8 Temperature revisited
  - 5.9 Mass balance revisited
  - 5.10 Impact on [P3045R7]
  - 5.11 Implementation status
- 6 Affine spaces within quantity hierarchies
  - 6.1 Motivation
  - 6.2 The `point_for<>` attribute
  - 6.3 Extended quantity specification arithmetic
  - 6.4 Precise convertibility for point-annotated quantities
    - 6.4.1 Valid reference forms

- 6.4.2 Arithmetic and conversion
- 6.5 Why `quantity<point<R>>` replaces `quantity_point<R>`
- 6.6 Implementation status
- 6.7 Standardization plan
  - 6.7.1 `quantity<point<R>>` unification
  - 6.7.2 `point_for<>` attribute
- 7 Range-validated quantity points
  - 7.1 Overview
  - 7.2 Design rationale
    - 7.2.1 Constraints belong to the measurement space, not to enforcement parameters
    - 7.2.2 Why not model bounded variants as distinct quantities in the hierarchy?
    - 7.2.3 Bounds values are deltas, not points
  - 7.3 Overflow policies
  - 7.4 Automatic bounds for non-negative quantity specifications
  - 7.5 Example: angular coordinates
  - 7.6 Hierarchical bounds validation
  - 7.7 Always-on enforcement with `constraint_violation_handler<Rep>`
  - 7.8 Interaction with `std::numeric_limits`
- 8 Runtime frame projections
  - 8.1 Motivation
  - 8.2 Comparison with `relative_point_origin`
  - 8.3 Proposed customization point
  - 8.4 Integration with `point_for()`
  - 8.5 Bidirectional projections
  - 8.6 Bounds on projected results
  - 8.7 Stateless example: altitude ↔ depth
  - 8.8 Stateless example: geometric azimuth ↔ bearing
  - 8.9 Runtime arguments: camera calibration
  - 8.10 Parameter-dependent conversion: geoid undulation
  - 8.11 Projection requirements and failure modes
  - 8.12 Scope limitations
- 9 Text output for quantity points
  - 9.1 Current state
  - 9.2 The challenge of default representations
  - 9.3 Resolution through absolute quantities

- 9.4 Recommendation for general points
- 10 Integer division safety
  - 10.1 Background
  - 10.2 The `operator%` asymmetry
  - 10.3 Current direction
  - 10.4 New information from the Au library
  - 10.5 Scope
- 11 Comparison against zero
  - 11.1 The problem
  - 11.2 Strategy 1: Named comparison functions (`is_eq_zero` etc.)
  - 11.3 Strategy 2: A Zero tag type
  - 11.4 Strategy 3: Literal `0` with compile-time guard
  - 11.5 Discussion and current status
- 12 Summary
- 13 Next steps
- 14 Acknowledgments
- 15 References

## § 1 Abstract

[P3045R7] proposes a C++ quantities and units framework built on two core abstractions: `quantity` (displacement vectors) and `quantity_point` (points in an affine space). Real-world experience with the [mp-units] reference implementation, together with feedback from SG6, BSI, ANSI, and the broader C++ community, shows that this two-abstraction model is insufficient for several common engineering patterns and, in places, mathematically too permissive.

This paper proposes a unified extension of the mathematical model underlying [P3045R7] along five axes: a compile-time **non-negativity** tag (a half-line / convex-cone structure distinct from the displacement-vector model), **absolute quantities** as a third arithmetic abstraction anchored at a true zero, **affine space annotations within the ISQ hierarchy**, **range-validated quantity points**, and **runtime frame projections**. It also resolves the **text output** problem for absolute quantities (the common case), discusses **integer division safety**, and surveys three candidate designs for **comparison against zero**.

This paper is the product of a WG21-requested convergence between two independently developed approaches: the [mp-units] reference implementation (Mateusz Pusz) and the Sequoia C++ library [Sequoia] (Oliver Rosten). SG6 asked both authors to work together toward a unified design rather than advance separate proposals. The mathematical framework of convex spaces [Rosten2025] developed by Oliver Rosten provides the theoretical foundation; implementation experience from both [mp-units] and [Sequoia] validates the structural conclusions. The two-axis extension described below is where their independent lines of work converge.

SG6 is asked to confirm the direction — specifically: whether absolute quantities should become the default meaning of `quantity<R>`, which integer division policy to adopt, and which zero-comparison strategy to standardize. Final wording review follows implementation.

## § 2 Introduction

[P3045R7] is a significant achievement. It models physical quantities using two class templates —

- `quantity` — represents a displacement vector in an affine space (a difference between two states), and
  - `quantity_point` — represents a point on a measurement scale, measured relative to an explicitly or implicitly defined origin.
- and provides six levels of type-safety: dimensional analysis, unit checking, representation type safety, quantity kind safety, quantity type safety, and mathematical space safety. A large category of physically meaningless operations is rejected at compile time. The bulk of this paper does not revisit those guarantees; it asks what further structure becomes expressible when the mathematical model is completed.

The goal is a library where the ideal gas law

```
quantity R_calc = P * V / (n * T);
```

compiles only when `T` is an absolute thermodynamic temperature — not a temperature difference or an offset-scale reading; where `mass` and `mass_loss` are distinct types the compiler prevents from being accidentally swapped; and where a function requiring a non-negative input carries that guarantee in its signature rather than as a runtime precondition buried in its body. That is the completed picture. The proposal describes the path from the current model to that destination.

The following gaps — identified through implementation experience in [mp-units] and committee feedback — stand between the current model and that goal. In each case the pattern is the same: *the runtime check is the symptom; the missing type-system structure is the disease*. The two-abstraction model is either *too coarse* (forcing users to encode physical structure as runtime preconditions) or *too permissive* (admitting operations the underlying physics forbids). [Motivation and scope](#) discusses each gap in detail; in summary they are:

- the **temperature trap** caused by offset units interacting with the affine model (§ [The temperature trap](#));
- the absence of any way to express that a quantity’s domain is the half-line  $[0, +\infty)$ , forcing every function to validate non-negativity at runtime (§ [Non-negative quantities lack a mathematical home](#));
- the inability to distinguish *amounts* (e.g., total mass) from *changes* (mass loss) in the type system (§ [Mass balance and accumulation](#));
- structural and runtime limitations of `relative_point_origin` (§ [Limitations of relative\\_point\\_origin](#));
- the lack of library support for **bounded domains** (§ [Bounded domains](#));
- the inability to format `quantity_point` values (§ [Points without output](#));

- ISQ position/displacement pairs that the library cannot represent as such, allowing physically meaningless operations to compile silently (§ [Affine relationships in quantity hierarchies](#));
- a silent **integer division** hazard when operand units are non-equivalent (§ [Integer division hazard](#)); and
- the absence of a natural syntax for **comparison against zero** (§ [Comparison against zero](#)).

The remainder of the paper proposes solutions to each gap, drawing on the mathematical insights of an independently developed model based on *convex spaces* [Rosten2025] and synthesizing both lines of work into a single proposal that puts the quantities and units library on a firm mathematical foundation while remaining accessible to the broader C++ community.

Most proposed features are already implemented in [mp-units]. Two — absolute quantities (the three-way delta/absolute/point split) and affine space annotations within the ISQ hierarchy — are design-complete but not yet implemented in [mp-units]. The same structural model is, however, already implemented in Oliver Rosten’s Sequoia library [Sequoia] with a different API surface, so the theory is proven. Implementation in [mp-units] is planned before the next WG21 meeting in Brazil, following favorable SG6 direction. Those features involve breaking changes to [P3045R7] that cannot be introduced later without an API break, which is why early directional consensus is sought now rather than after full implementation. SG6 is free to adopt any subset of the proposed features independently — in particular, the non-negativity tag, range-validated points, and runtime frame projections do not depend on the absolute/delta split being resolved first.

## § 2.1 Design philosophy

This paper works from mathematical structure outward to API design. The starting point is measurement-theoretic classification of quantity spaces (vector spaces, affine spaces, convex spaces); design choices follow from that classification, validated against implementation experience in [mp-units] and concrete user feedback. Mathematical completeness and practical usability are treated as complementary requirements, not opposing forces.

[P3045R7] achieved consensus at Croydon 2026. This paper does not revisit that consensus — it extends it. The extensions proposed here are the natural next step that the consensus enables: completing the mathematical model that [P3045R7] begins, based on gaps identified through implementation experience and on WG21’s request that the two independent approaches be unified rather than advanced as competing proposals.

That said, the paper proposes extensions to [P3045R7] at a time when committee feedback reveals divergent perspectives. Some reviewers — notably Oliver Rosten (BSI) and Tiago Freire (ANSI) — argue that the library is *insufficiently expressive* for real-world use cases, lacking the mathematical completeness needed for production engineering applications. Other members of WG21 have expressed concern that the library is *already too complex*, and have advocated for simplification.

The tension reflects a genuine engineering trade-off: a library that is too simple fails to solve real problems; one that is too complex becomes harder to learn and teach, and may overwhelm non-experts who need only basic functionality. This paper seeks a pragmatic middle ground:

- **Address real gaps systematically.** Each proposed extension (§ [Motivation and scope](#)) is motivated by the mathematical structure of the underlying quantities — vector spaces, affine spaces, convex spaces — and validated against concrete use cases from the [mp-units] reference implementation and user feedback.

- **Maintain conceptual coherence.** The extensions form a unified mathematical model grounded in measurement theory (ratio scales, interval scales, affine spaces, convex spaces) rather than ad-hoc patches.
- **Minimize API surface growth.** Most features are pure additions that do not affect existing code. The single breaking change (absolute quantities as default) aligns the library’s defaults with physical intuition, reducing the teaching burden for new users.
- **Don’t pay for what you don’t use.** Most users will encounter two core abstractions: absolute quantities (amounts anchored at a true zero) and deltas (signed differences), since subtracting two absolutes yields a delta that requires explicit conversion if an absolute result is needed. Points remain fully optional — needed for offset coordinate systems (temperature scales, time zones, reference frames) and for modeling measurement readings relative to a specific origin. Advanced features like bounds checking, frame projections, and range transformations become invisible infrastructure when incorporated into domain-specific systems by a single expert: every downstream user gains compile-time safety with every function call, without needing to understand the underlying mechanisms. Complexity scales with user sophistication, ensuring that the cost is proportional to requirements.
- **Preserve extensibility.** The design deliberately avoids closing off a more complete treatment of additional abstractions in a future revision. The three core class templates — `quantity<R>`, `quantity<delta<R>>`, and `quantity<point<R, 0>>` — are open for additional reference-wrapper specializations. The bounds policy mechanism (`check_non_negative`, `check_in_range`, and their siblings) is user-extensible: any type satisfying the policy concept can be attached to an origin as an NTP. Nothing in this proposal forecloses a follow-on paper that introduces a dedicated `convex<R>` reference wrapper or extends the non-negativity machinery to handle bounded domains (Celsius, bearing, opacity) with the same first-class status as the current half-line abstraction. Logarithmic quantities and units — decibels (dB), neper (Np), pH, stellar magnitude — are another natural extension point: they require a distinct arithmetic structure, but fit naturally into the reference-wrapper model as a future `log<QS>` quantity spec specialization, where the reference level, logarithmic base, and scale multiplier are encoded in the named unit (e.g., dBm, dBW, Np), without any change to the core quantity class template. A complementary model proposed by Anthony Williams [Williams2025] treats every measurement as carrying an explicit **anchor point** as part of its unit, deriving arithmetic compatibility from whether anchor points are compatible. That approach maps naturally onto the same extension mechanism: a future `quantity<anchored<R, Anchor>>` reference wrapper could express Williams’ anchored quantities without any change to the core quantity class template.

The goal is not to satisfy every theoretical desideratum, but to provide a library that is *usable* by domain experts (physicists, engineers, control systems programmers) while remaining *learnable* by C++ developers encountering quantities and units for the first time.

No mainstream units library in any programming language — including F# Units of Measure, Haskell’s *dimensional* package, or Python’s Pint — distinguishes absolute quantities, deltas, and points as three separate types in the type system. The closest prior art is Oliver Rosten’s Sequoia C++ library [Sequoia], which independently explores the same three-way split from a mathematical perspective. The convergence of two independent implementations on the same structural conclusions strengthens the case that this taxonomy reflects genuine mathematical structure rather than an

arbitrary design choice. No production-scale validation of the full three-way split exists yet in any library; this paper closes that gap.

[ *Note: **Author disclosure.*** The author is the creator and maintainer of the [\[mp-units\]](#) reference implementation cited throughout this paper as evidence for design decisions. Readers should weigh implementation citations with that context in mind. — *end note* ]

## § 3 Motivation and scope

This section identifies nine categories of issues relevant to the quantities and units library design. The first seven describe real-world use cases where [\[P3045R7\]](#)’s two-abstraction model is insufficient or forces users into awkward workarounds. The eighth and ninth (integer division safety and comparison against zero) are not direct limitations of the two-abstraction model, but are included here because they are highly relevant to SG6 and benefit from recent implementation experience. Each subsection provides concrete examples from the [\[mp-units\]](#) reference implementation and user feedback. The subsequent sections ([Non-negative quantities–Comparison against zero](#)) propose solutions to address these gaps.

### § 3.1 The temperature trap

By far the most frequently reported usability issue with the current [\[P3045R7\]](#) design involves temperature. The root cause is the interaction between offset units (°C, °F) and the affine space model.

Consider the ideal gas law,  $P V = n R T$ . A user might naively attempt:

```
auto P = 1. * atm;
auto V = 1. * L;
auto n = 1. * mol;
auto T = 28. * deg_C;                // x does not compile in P3045!
auto R_calc = P * V / (n * T);
```

This code does **not compile** under [\[P3045R7\]](#). The multiply syntax (`28 * deg_C`) was initially allowed in earlier revisions but was disabled after feedback from Tiago Freire, who demonstrated the danger with a concrete ideal gas law example. The fix was to disable the multiply syntax for **all** temperature units — preventing any of them from being used in a value `* unit` expression. The implementation mechanism for this is that every temperature unit using a point origin in its definition is caught by the restriction; Kelvin was therefore defined with `absolute_zero` as its point origin specifically to bring it under the same rule, ensuring that `28 * K` does not compile under [\[P3045R7\]](#) either.

The inclusion of Kelvin is necessary for a consistent and safe rule. The multiply syntax produces a quantity (delta), so `28 * K` would not itself construct a `quantity_point`. For Kelvin this causes no direct harm, but the situation changes entirely if the unit in such an expression were later changed to an offset unit (e.g., `deg_C`) — the result would silently become a meaningless delta rather than a compile error. A uniform rule — no multiply syntax for any temperature unit — is the only way to make the restriction predictable and refactoring-safe.

Nevertheless, this is fundamentally an **ad-hoc patch** applied on top of the displacement-vector model rather than a principled solution from measurement theory. It is the right fix for the model as it stands, but it reveals a deeper inadequacy: the model has no way to express “an absolute thermo-



dynamic temperature of 28 K” directly. The constraint is well-motivated to someone who understands the affine space chain, but may appear over-constraining to others.

Users must explicitly choose between a delta and a point:

```
auto T1 = delta<K>(28);           // a temperature difference of 28 K
auto T2 = point<K>(28.);          // 28 K on the Kelvin scale (= 28 K from
    absolute zero)
```

Neither option correctly captures the intent. By using `delta<K>`, `T1` is typed as a *displacement* — a signed difference between two temperatures — which is semantically misleading in  $PV = nRT$ , where 28 K is an absolute thermodynamic temperature, not a difference between two temperatures. Deltas may also be negative (e.g., `delta<K>(-5)` is a perfectly valid temperature difference of  $-5$  K), yet 28 K as a thermodynamic temperature can never be negative — a property that `delta<K>` gives no way to express or enforce. The point `T2` carries the right physical meaning but cannot be multiplied or divided, as point arithmetic in an affine space forbids those operations.

The workaround that produces a correct result requires constructing a point measured in Kelvin and extracting the displacement from absolute zero:

```
auto temp = point<deg_C>(28.);
auto T = temp.in(K).quantity_from_unit_zero();    // 301.15 K - correct but
    cumbersome
auto R_calc = P * V / (n * T);
```

While the affine space model *can* handle this case, the ergonomics are poor: `point<deg_C>(28.).in(K).quantity_from_unit_zero()` is a far cry from what a domain expert would consider natural. The need to go through a point, convert units, and then extract the displacement from zero is a significant usability burden that discourages correct usage.

An alternative approach — suggested by Chip Hogg as a “nature-based constant” idiom — is to subtract `absolute_zero` directly from the point, obtaining the displacement from the physical zero:

```
auto temp = point<deg_C>(28.);
auto T = temp - absolute_zero;           // 301.15 K - displacement from
    absolute zero
auto R_calc = P * V / (n * T);
```

This works even when the point is expressed in `deg_C`: the subtraction of the absolute zero origin yields a delta whose value is the absolute thermodynamic temperature, and the result can always be scaled to whatever unit is needed. However, this idiom still requires users to know that `absolute_zero` exists, understand why it must be subtracted, and remember to apply it every time an absolute temperature is needed — a teaching burden that grows with every new domain that has an analogous pattern.

This problem is not limited to temperature — it affects any domain where absolute quantities (as opposed to differences) are the natural operands in physical equations. The [hardware voltage example](#) in the [\[mp-units\]](#) documentation illustrates the same pattern in embedded systems: an ADC maps a physical voltage range to integer counts, and the measured voltage is an absolute scalar quantity, not a difference. Users have independently requested the same ergonomic improvements for such cases (see [mp-units discussion #606](#)). The solution, described in [Temperature revisited](#), changes the very definition of the Kelvin unit so that `28 * K` creates an absolute thermodynamic temperature directly, without any conversion idiom.

## § 3.2 Non-negative quantities lack a mathematical home

The displacement-vector model in [P3045R7] treats every quantity as an element of a one-dimensional vector space over  $\mathbb{R}$ . This is the right abstraction for *signed differences* — temperature differences, velocity changes, accumulated drift, signed power flow — and it is exactly what the affine-space machinery requires.

Many physical quantities, however, do not live in such a vector space at all. *Mass, length, duration, thermodynamic temperature, amount of substance, luminous intensity, kinetic energy, speed, area, volume, absorbed dose*, and the entire family of *power ratios* (linear or logarithmic) all share the same algebraic structure: their physical domain is the **half-line**  $[0, +\infty)$ . The half-line is not a vector space — it is a **convex cone**, closed under addition and under non-negative scalar multiplication, but **not** under negation. Independent work on convex-space foundations for measurement [Rosten2025] reaches the same conclusion: a quantity<isq::mass[kg]> is mathematically *not* an element of a vector space, and modelling it as one is a category error, not just an ergonomic one.

Encoding a half-line value as a member of a vector space has two practical consequences:

- **The model is too permissive.** Operations that the physical domain does not support — unary negation, signed addition without a reference point, signed scalar multiplication by a negative number — compile and run silently. A computation that ought to be rejected statically (a structural type error) instead becomes either undetected nonsense or a runtime contract violation that fires deep inside an arithmetic expression.
- **Every API boundary becomes a precondition site.** Because the type system carries no evidence that an `isq::mass`, an `isq::duration`, or a `kinetic_energy` is non-negative, every function that depends on this fact must re-validate it on entry. The check is the symptom; the missing structure is the disease.

The runtime cost of preconditions is the most visible symptom and is described first; the deeper structural argument is what motivates encoding non-negativity in the type system rather than as a wrapper of contracts.

### § 3.2.1 Preconditions as a symptom

Many physical functions naturally expect non-negative quantities. Without library-level support for the half-line structure, users must manually validate inputs with contract preconditions to ensure correctness. Consider a canonical physics function whose inputs are unambiguously non-negative — electrical energy, where both power and duration are inherently amounts, not differences:

```
quantity_of<isq::energy> auto electrical_energy(quantity_of<isq::power> auto
    power,
                                                quantity_of<isq::duration> auto
    duration)
{
    // Manual precondition checks – user's responsibility
    MP_UNITS_EXPECTS(power >= 0 * W);
    MP_UNITS_EXPECTS(duration >= 0 * s);

    return power * duration;
}
```

These precondition checks present several problems:

- **Repetition**: every function must validate its own inputs.
- **Fragility**: easy to forget or apply inconsistently.
- **Runtime overhead**: checks execute even when inputs are known to be valid at the call site, and they accumulate across every function in a call chain.
- **Hidden rescaling cost**: the `0 * unit` check silently introduces a unit conversion when the spelled unit differs from the quantity’s stored unit (e.g., `duration >= 0 * s` when the value is stored in min). The comparison requires rescaling both sides to a common unit before the numerical comparison — yet mathematically this is unnecessary, because zero is zero in every unit and no conversion is needed at all. This issue is discussed further in [Comparison against zero](#).
- **Unclear intent**: the type system cannot distinguish a signed delta from a non-negative amount, so reviewers and tooling have no way to tell which guarantee a given parameter carries.

Most physical equations (unless explicitly working with vector quantities or differences marked with  $\Delta$ ) assume non-negative operands. The square root of kinetic energy, gravitational force from two masses and a distance, density from mass and volume — all expect non-negative inputs. Users shouldn’t need to guard every function with manual checks when the mathematics itself requires non-negativity.

The overhead problem compounds when results flow through pipelines:

```
quantity area      = rectangular_area(width, height);    // validates width and
    height
quantity volume    = box_volume(area, depth);            // must validate area and
    depth again
quantity rho       = mass_density(mass, volume);        // must validate mass and
    volume again
```

Each function must guard its inputs independently. The type of `area` is `quantity<isq::area>` — indistinguishable from a quantity that could be negative. Without that information in the type system, `box_volume` cannot know that `area` was produced by a call that guarantees non-negativity, so it must check unconditionally. These checks are not a user oversight — they are *required* by the absence of type-level guarantees. The only way to avoid them today is to skip them entirely and accept the risk.

## § 3.2.2 Why the structure matters more than the checks

If non-negativity were merely a runtime property, a thin contract-checked wrapper would suffice. But the half-line is a different mathematical object than the real line, and adequately representing it requires meeting requirements that a wrapper cannot satisfy:

- *Propagation through arithmetic*. A non-negativity guarantee should survive arithmetic: the sum, product, quotient, and positive integer power of non-negative quantities are themselves non-negative. Conversely, once any operand can be negative, the result may legitimately lose the guarantee. A contract-checked wrapper re-validates at every call site — it has no way to distinguish “guaranteed non-negative by construction from non-negative operands” from “might be negative.” Propagation rules also cannot automatically extend to named root derived quantities — the reasons are varied: reactive power has all-non-negative dimensional factors ( $V \cdot A = W$ ) yet is physically signed because the phase angle carries the sign, invisible to dimensional analysis; the Massieu function is defined with explicit negation ( $J = -A/T$ ) that dimen-

sional structure does not capture; Helmholtz and Gibbs energies share the dimension of energy with `kinetic_energy` yet are possibly-negative thermodynamic potentials. Any propagation mechanism must therefore require an explicit annotation at every named root derived spec.

- *Correct arithmetic abstraction.* On the half-line, addition and non-negative scaling are closed operations (the result always stays on the half-line), but unary negation is not. Treating mass, duration, `kinetic_energy` as displacement vectors makes `-m` type-check silently, even though a negative absolute mass has no physical meaning. Even the seemingly reasonable `mass_a - mass_b` is problematic: the result can be negative, yet it carries the same type as an absolute mass and can then silently be passed to any function that expects a non-negative amount. The type system needs a way to express whether a quantity lives on the half-line or the full real line — and to enforce that distinction through arithmetic.
- *Checks at boundaries, not inside expressions.* Non-negativity checks belong at *visible* entry points — when constructing a quantity from a raw number, converting from a signed delta, or reading from external input — not scattered as hidden runtime checks through every arithmetic expression. For this to work, arithmetic on already-validated quantities must not require re-checking; the guarantee has to be carried by the type, not repeated at every operation.

These requirements cannot be met by a wrapper or a coding convention; they call for non-negativity to be part of the type system. The solution is described in [Non-negative quantities](#), and the absolute-quantity abstraction that gives the half-line a value-level home follows in [Absolute quantities](#).

## § 3.3 Mass balance and accumulation

In process engineering, it is common to track the total mass of material in a vessel, then compute a percentage loss:

```
quantity<percent> moisture_loss(quantity<kg> water_lost, quantity<kg> total)
{
  MP_UNITS_EXPECTS(total >= 0 * kg);
  return water_lost / total;
}
```

Here, `water_lost` is a difference (delta) and `total` is an absolute amount, but the type system makes no distinction — both are `quantity<kg>`. A user could accidentally swap them:

```
quantity result = moisture_loss(total_initial, total_initial - total_dried);
// Compiles. Wrong.
```

One might attempt to address this by defining specialized quantities within the `isq::mass` hierarchy — for example, `total_mass` and `mass_loss` as children of `isq::mass`. The library’s quantity hierarchy does support this: specialized child quantities of the same kind can be subtracted and compared, but implicit conversion from a parent quantity to a child requires an explicit cast. This would indeed prevent accidental swapping of the two arguments, since `total_mass` and `mass_loss` would be distinct types. However, it also means that the legitimate computation `total_initial - total_dried` yields an `isq::mass` result — not a `mass_loss` — and passing it to `mass_loss` would require an explicit conversion. The workaround thus introduces friction for every correct call site, not just the incorrect ones.

However, the distinction here is not between different *specializations* of mass. Both `water_lost` and `total` may even be measured with the same instrument, but the *measurement setup* is fundamentally different: `total` is a direct ratio-scale reading anchored at true zero (put the wet material

on the scale and read the mass), while `water_lost` is derived as the difference between two such readings. The same scale, a completely different kind of measurement. The difference is in their *role*: one is a signed difference (delta) and the other is an absolute amount measured from true zero. Encoding every such role as a separate quantity in the hierarchy would cause the quantity tree to proliferate with domain-specific entries that have no basis in measurement science (ISO 80000 does not distinguish “total mass” from “mass loss” — they are both simply *mass*). Every domain would need its own set of role-based quantity specializations, and the hierarchy would grow without bound.

If instead the library distinguished absolute quantities (amounts measured from a true zero) from deltas (signed differences), the function signature alone would enforce the correct usage without polluting the quantity hierarchy. The solution is shown in [Mass balance revisited](#).

## § 3.4 Limitations of `relative_point_origin`

The `relative_point_origin<QP>` design in [\[P3045R7\]](#) is a powerful and elegant mechanism for expressing compile-time relationships between coordinate systems, measurement scales, and reference frames. It enables type-safe conversions between origins whose offset is known at compile time — such as converting between Celsius and Kelvin, or between epoch-based time representations.

However, this design has several fundamental limitations that prevent it from addressing certain real-world use cases:

### § 3.4.1 Structural type requirements

C++20 restricts non-type template parameters to *structural types* — class types where **all non-static data members must be public** and themselves of structural type. This excludes many `constexpr`-friendly user-defined types:

- **Decimal and fixed-point types** that use private members to maintain invariants
- **Rational types** with private numerator/denominator to enforce normalization
- **Arbitrary-precision types** with private dynamically-sized storage
- **Any type with encapsulation** — even a simple `class` `Distance` with a private `double` `value_` member cannot be structural

These types work perfectly in `constexpr` contexts and can represent compile-time-known values, but the **public members requirement** makes them unusable as NTTPs. Until a mechanism like [\[P3380R1\]](#) (reflection-based structural type opt-in) is adopted, there is no way to use such types with `relative_point_origin` even when the offset is known at compile time.

### § 3.4.2 Parameter-dependent and runtime-determined origins

Even when the representation type is structural, many real-world use cases require origins whose relationship depends on runtime parameters or state:

- **Location-dependent altitude conversions**: Converting between mean sea level (MSL) and height above ellipsoid (HAE) requires the geoid undulation, which varies with geographic position. A GPS waypoint at (54.25°N, 18.67°E) has a different MSL↔HAE offset than one at

(37.77°N, 122.42°W). The origins themselves (MSL, HAE) are compile-time types, but the transformation between them depends on GPS coordinates known only at runtime.

```
// unmanned_aerial_vehicle.cpp – today's workaround
hae_altitude to_hae(msl_altitude msl, position<double> pos)
{
    quantity undulation = geoid_undulation_at(pos.lat, pos.lon); // depends on
    location
    return height_above_ellipsoid + (msl - mean_sea_level - undulation);
}
```

- **Multi-joint robot arm:** In a robotic arm with  $N$  joints, the position of the end-effector relative to the world frame is the composition of  $N$  per-joint frame transformations. Each transformation depends on the joint's current angle, read from encoders at runtime. `relative_point_origin` can only express offsets that are compile-time constants, so it cannot represent any of these transformations — not even the relationship between two adjacent joint frames. There is no library mechanism to express or verify the chain from the end-effector back to world coordinates.

These cases require runtime parameters or state to determine the relationship between origins, which cannot be expressed as a compile-time NTTP offset in `relative_point_origin`.

### § 3.4.3 Scale, translation, and rotation

Oliver Rosten has raised in LEWGI discussions that the current `relative_point_origin` design can express **scaling** (unit conversions) and **translation** (constant offsets), but not **rotation** or other affine transformations. Coordinate frame transformations in robotics, computer vision, and geospatial applications often require full affine mappings — not just shifts along a single axis.

The multi-joint robot arm described in the previous subsection is a direct illustration: even setting aside the runtime-determined joint angles, each per-joint transformation involves a rotation in 3D space. `relative_point_origin` can only shift along a single axis, so it cannot represent any step in the kinematic chain — regardless of whether the angle is known at compile time or not.

A simpler example of the same class of limitation is **axis inversion**: converting between altitude (measured upward from sea level) and depth (measured downward from the ocean surface) requires negating the value, which cannot be expressed with `relative_point_origin`. As discussed in [#782](#), one quantity needs to become an inversion of another, and a user-provided projection function can encode such relationships.

Two geographic coordinate examples illustrate when `relative_point_origin` is and is not sufficient:

- **Heading azimuth:** This uses  $0^\circ$  = North but increases counter-clockwise. The relationship `heading = azimuth - 90°` involves only translation (no sign flip), so `north_ccw` can be implemented as `relative_point_origin<east - 90°>`.
- **Geometric azimuth vs. bearing:** Bearing ( $0^\circ$  = North, increasing clockwise) differs from geometric azimuth ( $0^\circ$  = East, increasing counter-clockwise) by both a shift *and* a sign flip: `bearing = 90° - azimuth`. Unlike the heading case where the constant is on the right (`azimuth - 90°`), here the variable is subtracted *from* the constant, inverting the axis direction. This cannot be expressed with `relative_point_origin`, so `north_cw` (bearing origin) must be defined as



an independent `absolute_point_origin`, severing the compile-time relationship with `east` (geometric azimuth origin).

The practical consequence is that quantity points expressed in bearing (`north_cw`) and quantity points expressed in geometric azimuth (`east`) become **incompatible types** — the library cannot convert between them, even though both represent the same physical angle on the same circle. A function that accepts a bearing cannot receive a geometric azimuth, and vice versa, without a user-written explicit conversion that encodes the  $90^\circ$  – azimuth formula.

```
// origin definitions
inline constexpr struct east : absolute_point_origin<isq::angular_measure> {}
    east;
inline constexpr struct north_ccw : relative_point_origin<east - 90 * deg> {}
    north_ccw;
inline constexpr struct north_cw : absolute_point_origin<isq::angular_measure> {}
    north_cw;

using geometric_azimuth = quantity_point<deg, east>;
using heading           = quantity_point<deg, north_ccw>;
using bearing           = quantity_point<deg, north_cw>;

void navigate(heading h);

geometric_azimuth az = east + 30 * deg;
heading           h  = north_ccw + 30 * deg;
bearing           b  = north_cw + 30 * deg;

navigate(h); // ✓ compiles – same type
navigate(az); // ✓ compiles – implicit conversion, north_ccw is relative to east
navigate(b); // ✗ does not compile – no path from north_cw to east, no projec-
    tion defined
```

Addressing these cases requires introducing a `frame_projection` mechanism that can express arbitrary geometric transformations between reference frames — including rotation, reflection, and non-uniform scaling — that are essential for multi-axis coordinate systems. This mechanism is proposed in [Runtime frame projections](#).

## § 3.5 Bounded domains

Many physical quantities have natural bounds:

- Angular coordinates: longitude  $\in [-180^\circ, 180^\circ)$ , bearing  $\in [0^\circ, 360^\circ)$
- Thermodynamic temperature:  $T \geq 0$  K
- Sensor operating ranges: pressure  $\in [100, 1100]$  hPa

Two approaches to enforcing bounds present themselves, and both fall short.

The first is to delegate bounds enforcement to the representation type — a user-defined numeric type whose constructor or assignment operator rejects out-of-range values. This seems appealing, but it cannot work: the representation type is unit-unaware. A `BoundedDouble<0, 360>` rep type intended to enforce the bearing range  $[0^\circ, 360^\circ)$  works correctly when the quantity is stored in degrees, but becomes meaningless when stored in radians: all valid radian values fall in  $[0, 2\pi] \approx [0, 6.28]$  — well within the  $[0, 360]$  numeric bound — so an out-of-range bearing of 7 rad ( $\approx 401^\circ$ ) is

silently accepted, and the check enforces nothing. The representation type has no access to the unit, so it cannot reason about physical bounds.

The second approach is to write quantity-level precondition checks — comparisons such as `b >= 0 * deg && b < 360 * deg`. Here unit scaling is not an issue: the library automatically converts both sides to a common unit before comparing, so the check is correct regardless of whether `b` is stored in degrees or radians. But this brings the same problems already described in [Manual precondition checking](#): the checks are repetitive, error-prone, and carry no type-level guarantee — callers downstream cannot know that the value was already validated and must check again. Furthermore, for cyclic domains such as bearing, every arithmetic result must be manually wrapped back into valid range: adding two bearings, negating one, or computing a midpoint can all produce an out-of-range value, and the library provides no way to enforce wrap-around automatically. Every operation that produces a new bearing requires its own hand-written modulo reduction. The proposed solution is described in [Range-validated quantity points](#).

## § 3.6 Points without output

The current library deliberately omits text output for `quantity_point`. Users have consistently requested this feature — the need is real. Domain experts working with altitudes, timestamps, geographic coordinates, or sensor readings want to print measurement results directly:

```
quantity_point<m, mean_sea_level> alt = mean_sea_level + 1350 * m;
// std::println("{} ", alt); // Does not compile today
std::println("{} AMSL", alt.quantity_from(mean_sea_level)); // Current workaround
```

However, providing a default text representation for quantity points is non-trivial. Unlike units, which have standard symbols, points have no universal notation. The same physical point can be expressed relative to different origins, and it's unclear what a generic library should print by default. How absolute quantities resolve the common case, and what is recommended for general points, is discussed in [Text output for quantity points](#).

## § 3.7 Affine relationships in quantity hierarchies

The ISQ defines several pairs of quantities where one naturally behaves as a “position” and the other as a “displacement.” The clearest example is `position_vector` and `displacement`: subtracting two positions should yield a displacement, and adding a displacement to a position should yield a position.

But the library has no way to express this relationship. Given two position vectors:

```
quantity pos1 = isq::position_vector(vector{1, 2} * m);
quantity pos2 = isq::position_vector(vector{2, 3} * m);
quantity q1 = pos2 - pos1; // result is position_vector, not displacement!
quantity q2 = pos1 + pos2; // compiles – but adding two positions is meaningless
```

The result of subtraction has type `position_vector`, because that is the common quantity type of the two operands — the standard tree-walking rules have no concept of point/delta pairing. Worse, addition of two position vectors compiles without warning, even though the operation is physically nonsensical — adding “where object A is” to “where object B is” has no geometric interpretation.



The same issues arise with scalar quantities. Subtracting two altitudes should yield a height, but the library returns an altitude:

```
quantity alt1 = isq::altitude(100. * m);
quantity alt2 = isq::altitude(120. * m);
quantity q1 = alt2 - alt1; // result is altitude, not height!
quantity q2 = alt1 + alt2; // compiles – but adding two positions is meaningless
```

The expression `alt1 + alt2` is particularly dangerous: altitude is a position-like quantity (a “where”), and adding two positions has no physical meaning. Yet the library permits it because it sees two quantities of the same type and applies ordinary addition.

There is a deeper problem here. The library’s quantity hierarchy must encode the relationship between `position_vector` and `displacement` somehow, but every option available within [P3045R7]’s current type system has significant drawbacks:

### Option 1 — Parent–child (current [mp-units] V2 design).

Make `position_vector` a child of `displacement`:

```
length[m]
└─ displacement{vector}
   └─ position_vector{vector}
```

This encoding captures the intuition that every position vector “is a kind of” displacement, allowing implicit conversion upward. However, the default arithmetic rules produce incorrect results:

- `pos2 - pos1`: Both operands are `position_vector`, so the common type is `position_vector` — not `displacement`. The result is implicitly convertible to `displacement` (child to parent), so assignment works, but the deduced type is wrong and the semantic intent is obscured.
- `pos + disp`: The common type of `position_vector` (child) and `displacement` (parent) is `displacement` — not `position_vector`. Adding a displacement to a position should yield a position, but the result requires an explicit conversion back to `position_vector`.

The addition case defeats the purpose of type-safe arithmetic: every `pos + disp` expression requires a manual explicit conversion.

### Option 2 — Reversed parent–child.

Swap the direction: make `displacement` a child of `position_vector`. Applied to vectors, the tree would be:

```
length[m]
└─ position_vector{vector}
   └─ displacement{vector}
```

Now `pos + disp` yields `position_vector` (correct), but `pos2 - pos1` still yields `position_vector` (wrong — should be `displacement`), and the result is *not* implicitly convertible to `displacement` (parent to child requires an explicit conversion). The reversed hierarchy also misrepresents the physical relationship: not every displacement “is” position vector, and the implicit conversion from `displacement` to `position_vector` is physically unsound.

### Option 3 — Aliases (current [mp-units] V2 design for altitude/height).

Define one as an alias of the other (`auto position_vector = displacement`) — making them literally the same type. This is the approach [mp-units] V2 uses for altitude and height: they are defined as aliases of the same quantity. This mirrors ISO 80000 definitions for those quantities. This also eliminates the arithmetic problem entirely (there is only one type), but also eliminates the semantic distinction. Function signatures, error messages, and type constraints can no longer differentiate positions from displacements. The two quantities become interchangeable, which is exactly the kind of confusion a type-safe library should prevent. There is also an unavoidable naming problem: two object identifiers (altitude and height, position\_vector and displacement) map to a single type identifier, and there is no principled basis for choosing which name the type should carry. Whichever name is chosen, the other becomes a mere alias with no independent presence in the type system — it disappears from diagnostics, concept error messages, and overload resolution alike.

#### Option 4 — Siblings (ISO-correct, unannotated).

Place both quantities on independent branches of the length hierarchy, matching the natural ISQ structure:

```
length[m]
├─ path_length
│   └─ distance
│       ├── radial_distance
│       │   └─ position_vector{vector}
│       └─ displacement{vector}
```

This layout has a unique advantage over the other options: it correctly determines the magnitude of each vector quantity. In the library, the magnitude of a vector quantity is its first scalar ancestor in the tree. Here, `norm(position_vector)` is `radial_distance` and `norm(displacement)` is `distance`. In Options 1 and 2, both vector quantities share a single parent branch, so the magnitude of `position_vector` degenerates to `length` (or `displacement`'s scalar parent), losing the specific scalar quantity in the ISQ hierarchy. Option 3 eliminates the distinction entirely, and Option 5 isolates the quantities so that magnitude relationships to the rest of the hierarchy are severed.

However, with the standard tree-walking rules alone, sibling quantities have no direct arithmetic relationship. `pos2 - pos1` yields `position_vector` (the common type of two identical quantity specifications), and converting it to `displacement` requires a `quantity_cast` since the two are on different branches. Meanwhile, `pos + disp` is a compile-time error: the first common ancestor of `position_vector` and `displacement` is `distance`, a scalar quantity, and assigning a vector quantity to a scalar quantity is ill-formed.

#### Option 5 — Distinct kinds (`is_kind`).

Mark `position_vector` as a separate quantity kind. This completely isolates it from `displacement` — no implicit conversion, no shared arithmetic, no comparison. Any interaction requires explicit conversion to a common base quantity, which is even more cumbersome than `quantity_cast` in Option 1 and blocks natural expressions like `pos + disp` entirely.

None of these options produces correct arithmetic without either manual casts or loss of type safety. The same pattern repeats throughout the ISQ: altitude and height, time and duration, and potentially many user-defined quantity pairs exhibit the same point/delta structure. The `point_for<>` attribute proposed in [Affine spaces within quantity hierarchies](#) resolves this by annotating the sibling layout of Option 4 with an explicit point/delta pairing, enabling the library to enforce correct affine arith-

metic (point – point = delta, point + delta = point, point + point = ill-formed) without casts, hierarchy inversions, or loss of type information.

## § 3.8 Integer division hazard

The `/` operator is an **arbitrary-unit operation**: the raw stored values are divided as integers and the units are composed, without any prior normalization. This rule applies uniformly across all quantity pairs, regardless of whether their dimensions or units are related:

```
quantity q1 = (8 * km) / (40 * min); // quantity<km/min, int>, stored value = 8 /
40 = 0
quantity q2 = (8 * h) / (40 * h);    // quantity<h/h, int>, stored value = 8 /
40 = 0
quantity q3 = (8 * h) / (40 * min);  // quantity<h/min, int>, stored value = 8 /
40 = 0
```

For q1, the operands have different dimensions (length and time), so there is no common unit to convert to — arbitrary-unit division is the only option. For q2, the units are identical and the integer truncation is obvious to any C++ engineer. For q3, the dimensions are the same but the units differ, yet the result is the same: the stored values 8 and 40 are divided directly.

Consistency across all three cases is the design intent. Making `/` normalize units before dividing for some quantity pairs but not others would produce unexpected behavior in generic code: a template operating on q2-like operands and q3-like operands must see the same semantics.

The potential surprise in q3 is that a user expecting normalization would anticipate a different result:

```
quantity<si::hour, int> work_time = 8 * h;
quantity<si::minute, int> break_time = 40 * min;
quantity ratio = work_time / break_time;
// quantity<h/min, int>, stored value = 8 / 40 = 0
// a user expecting normalization might anticipate 480 / 40 = 12
```

This behavior is a potential surprise rather than an error: the result is well-defined C++ integer arithmetic, and users who are aware of it can work with it intentionally. The proposed approaches and their trade-offs are analyzed in [Integer division safety](#).

## § 3.9 Comparison against zero

Comparing a quantity against zero is among the most frequent operations in physical computing. It appears in precondition checks, conditional logic, algorithmic guards, and invariant assertions throughout any codebase that works with measured values. Consider the examples already shown in [Manual precondition checking](#):

```
MP_UNITS_EXPECTS(power >= 0 * W);
MP_UNITS_EXPECTS(duration >= 0 * s);
```

The `0 * unit` syntax works, but it has a subtle cost: if the quantity’s stored unit differs from the spelled unit, the right-hand side must be rescaled before the numerical comparison. This is unnecessary — zero is zero in every (non-offset) unit, and no conversion is ever needed to establish that a quantity is non-negative. Yet the library has no way to detect that the right-hand side is exactly zero and elide the conversion.

The same pattern recurs wherever a quantity’s sign is tested:

```
// Guard before a physics formula
if (temperature > temperature.zero())
    entropy_change = heat_transferred / temperature;

// Clamp a velocity to non-negative (e.g., speed sensor floor)
if (measured_speed < 0 * m / s)
    measured_speed = 0 * m / s;

// Generic algorithm requiring non-negative input
template<quantity_of<isq::energy> auto Q>
void process(Q energy)
{
    assert(energy >= 0 * Q::unit);
}
```

Beyond the rescaling inefficiency, the `0 * unit` syntax requires knowing the unit at the comparison site, which breaks generic code. Several alternative designs have been proposed — a `.zero()` member, named comparison functions, a Zero tag type, or overloaded operators that accept a compile-time literal `0` — but each involves real trade-offs in expressiveness, genericity, or metrological clarity. There is no obviously correct answer. The alternatives and their trade-offs are analysed in [Comparison against zero](#).

## § 4 Non-negative quantities

### § 4.1 Overview

Oliver Rosten (BSI) [\[Rosten2025\]](#) raised non-negativity as an important property that the current [\[P3045R7\]](#) design lacks entirely: many physical quantities — length, mass, duration, thermodynamic temperature, amount of substance, luminous intensity — are inherently non-negative, and a library that cannot express this is forced to scatter manual precondition checks throughout user code (as shown in [Preconditions as a symptom](#)).

A complete solution would need to satisfy all of the following requirements:

- *Tagging*: a way to declare that a quantity spec’s physical domain is the half-line  $[0, +\infty)$ .
- *Inheritance*: child specs should inherit the guarantee from a tagged parent automatically, without any per-spec repetition.
- *Runtime enforcement*: constructing a value from a raw number, or converting from a signed type, should be checked at that boundary.
- *Propagation through arithmetic*: if a result is computed from non-negative operands, the result type should carry the guarantee automatically, so downstream code need not re-check.
- *Named derived specs*: a derived quantity such as area or speed, defined by an equation, should be able to carry the guarantee when its physics warrants it.

This chapter describes what [\[mp-units\]](#) V2 has already implemented (tagging, inheritance, and runtime enforcement on `quantity_point`) and where the design hits genuine limits (propagation through arithmetic and automatic inference for named derived specs). One open question for SG6 concerning negation is identified at the end of the chapter.

Throughout this chapter, `non_negative` is treated as a property of the *quantity specification*, not of individual values. A quantity-spec carrying the tag certifies that its physical domain is the half-line  $[0, +\infty)$ ; the library may then enforce that domain at appropriate boundaries depending on the abstraction in use.

## § 4.2 The `non_negative` tag

The tag is applied to any quantity spec whose physical domain is the half-line — commonly at the root of a hierarchy, but not exclusively. When a parent is tagged, the constraint propagates down to its children (see [Inheritance through the hierarchy](#) below). Conversely, a parent may be untagged (possibly-negative) while individual children carry the tag independently. Two important examples of the latter:

- **dimensionless** is possibly-negative (a pure ratio can be negative), but `mach_number`, `strain`, and other non-negative dimensionless children are tagged individually.
- **energy** is possibly-negative (thermodynamic potentials such as Helmholtz and Gibbs energy can be negative, and potential energy depends on the choice of reference), but `kinetic_energy` — which equals  $\frac{1}{2}mv^2$  regardless of reference — and similar children that are physically bounded below by zero are tagged individually.

```
inline constexpr struct length          : quantity_spec<dim_length, non_negative> {} length;
inline constexpr struct mass            : quantity_spec<dim_mass,    non_negative> {} mass;
inline constexpr struct duration        : quantity_spec<dim_time,    non_negative> {} duration;
// electric_current carries no tag – currents flow in either direction:
inline constexpr struct electric_current : quantity_spec<dim_electric_current> {} electric_current;
// energy carries no tag – thermodynamic potentials can be negative:
inline constexpr struct energy           : quantity_spec<...> {} energy;
// kinetic_energy is a child of energy but is tagged independently:
inline constexpr struct kinetic_energy   : quantity_spec<mechanical_energy, mass * pow<2>(speed), non_negative> {} kinetic_energy;
```

`is_non_negative(QS)` is the corresponding compile-time query.

## § 4.3 Inheritance through the hierarchy

Once `non_negative` is applied to a quantity spec, every named real-scalar descendant inherits the constraint unconditionally. There is no opt-out for children of a tagged parent, because every more-specialised quantity must sit inside the physical domain of its parent — every width, height, or radius is a length, so if length is non-negative, every specific real scalar type of length must be too. When the parent is *not* tagged, children may independently carry the tag (as with `kinetic_energy` under `energy`):

```
inline constexpr struct width  : quantity_spec<length> {} width;
inline constexpr struct height : quantity_spec<length> {} height;
inline constexpr struct radius : quantity_spec<width>   {} radius;
// width, height, radius all inherit non_negative from length (parent is tagged)

// energy is not tagged, but kinetic_energy is – no inheritance involved:
inline constexpr struct kinetic_energy : quantity_spec<mechanical_energy, mass * pow<2>(speed), non_negative> {} kinetic_energy;
```

```
static_assert( is_non_negative(isq::kinetic_energy));
static_assert(!is_non_negative(isq::energy));
```

Two categories of children are excluded from inheritance:

- **Quantities of vector, complex, or tensor character.** Such quantities are direction-sensitive or multi-component, making non-negativity physically meaningless. Inheritance is suppressed automatically, and an explicit `non_negative` tag on such a spec is a compile-time error:

```
// Vector child of length – inheritance is suppressed:
inline constexpr struct displacement : quantity_spec<length, quantity_character::vector> {} displacement;
static_assert(!is_non_negative(displacement));

// Compile-time error – non_negative is incompatible with vector character:
// inline constexpr struct bad : quantity_spec<length,
//   quantity_character::vector, non_negative> {} bad;
```

- **Point-like children annotated with `point_for<>`** (see [Affine spaces within quantity hierarchies](#)). A point on a scale can legitimately be negative even when the magnitude it locates cannot — altitude may sit below the reference origin, while height may not:

```
inline constexpr struct altitude : quantity_spec<length, point_for<height>> {} altitude;
static_assert(!is_non_negative(altitude));

// Compile-time error – point-like quantities cannot carry non_negative:
// inline constexpr struct bad : quantity_spec<length, point_for<height>,
//   non_negative> {} bad;
```

A `kind_of<QS>` instance is also never `non_negative`, even when QS itself is, because a kind represents the *entire* quantity tree rooted at QS, including its vector and signed descendants:

```
static_assert( is_non_negative(isq::length)); // ✓ tagged
static_assert(!is_non_negative(kind_of<isq::length>)); // ✗ kind covers signed subtypes
```

[ *Note*: The remainder of this section uses the three reference wrappers proposed in [Absolute quantities](#): `quantity<delta<Q>>` (a signed displacement, formerly the default quantity), `quantity<point<Q>>` (a point-valued quantity, replacing `quantity_point<Q>`), and a bare `quantity<Q>` (an absolute amount anchored at true zero). Their formal introduction and the motivation for absorbing `quantity_point` into a single quantity class template are in § [Absolute quantities — Syntax](#) and § [Why `quantity<point<R>>` replaces `quantity\_point<R>`](#). — *end note* ]

Similarly, `delta<Q>` is never `non_negative` regardless of Q, because a delta is a vector-space element with no fixed physical origin — it measures change, not amount from a true physical zero.

A `point<Q>` value (i.e., `quantity<point<Q>>[u]`, replacing the old `quantity_point`) carries the `non_negative` constraint **only** when Q is `non_negative` and the point is anchored at `natural_point_origin<Q>` — the physically meaningful true zero. Whether a user-defined origin also triggers the check depends on the kind of origin:

- A **relative\_point\_origin** that is defined relative to `natural_point_origin` (directly or transitively) inherits its check: the absolute value of the stored quantity — measured from the floor — must remain  $\geq 0$ , even though the offset origin allows negative relative values as long as they stay above the floor.

- An **absolute\_point\_origin** is an independent declaration that introduces its own zero for the scale without any relationship to `natural_point_origin`. It carries no auto-check, so negative values relative to it are accepted:

```
// mass is non_negative; point at natural origin → check fires:
quantity<point<isq::mass[kg]>> m = 10 * kg;    // OK
quantity<point<isq::mass[kg]>> bad = -5 * kg; // ✗ contract violation

// height is non_negative; average_height is a relative origin 1.7 m above the
// floor.
// Relative origins propagate the natural origin's check – absolute value must
// stay ≥ 0:
inline constexpr struct average_height : relative_point_origin<natural_point_ori-
    gin<isq::height> + 1.7 * m> {} average_height;

quantity<point<isq::height, average_height>[m]> h1 = -0.5 * m;    // OK – 1.2 m
    above floor
quantity<point<isq::height, average_height>[m]> h2 = -2.0 * m;    // ✗ – 0.3 m be-
    low floor

// mass is non_negative, but a laboratory balance reads its tare mass at runtime –
// the offset from true zero is not a compile-time constant, so
// relative_point_origin
// cannot be used. absolute_point_origin is the only option and carries no auto-
// check.
// Net readings below the tare (e.g., removing part of the container) are negative
// but valid:
inline constexpr struct tare_origin : absolute_point_origin<isq::mass> {}
    tare_origin;

quantity<point<isq::mass, tare_origin>[kg]> net1 = 2.5 * kg;    // OK – 2.5 kg
    above tare
quantity<point<isq::mass, tare_origin>[kg]> net2 = -0.3 * kg;    // OK – 0.3 kg be-
    low tare
```

The full mechanics — including how auto-attachment of `check_non_negative` to `natural_point_origin` works and how custom origins can supply explicit bounds — are described in [Runtime enforcement on quantity point](#) below.

## § 4.4 Why automatic inference cannot be implemented correctly

The `non_negative` flag lives exclusively on **named quantity\_spec objects**, as described in the previous chapters. Despite the fact that some `quantity_spec` definitions carry a quantity equation (their *defining recipe*), that equation cannot be used to infer the flag automatically, as shown below. A similar limitation applies to the intermediate results of user calculations: when an expression evaluates to a `derived_quantity_spec`, no check is performed — the library tests for the flag via a `requires` expression, and because `derived_quantity_spec` exposes no such member, the check is silently elided:

```
// Named specs: flag present or absent as explicitly stated
static_assert( is_non_negative(isq::mass));
static_assert( is_non_negative(isq::mass_density));
static_assert(!is_non_negative(isq::electric_current));
// Anonymous derived_quantity_spec – no non_negative member; requires expression
// fails:
// static_assert(!is_non_negative(isq::mass / isq::volume)); // ill-formed
```



The two failure modes are detailed below.

#### § 4.4.1 Inference from named spec definitions

A natural question is whether the library could infer `non_negative` for a named spec automatically from its defining equation.

A plausible inference rule would be:

**Factor-level rule (candidate).** A named spec defined as a product/quotient formula inherits `non_negative` if and only if every factor satisfies one of:

- its base spec carries `non_negative`; or
- it is a power  $Q^{N/D}$  where  $N/D$  is a positive even integer and  $Q$  has real-scalar character (not vector, complex, or tensor) — because  $x^{2k} \geq 0$  for any real  $x$ .

This rule correctly handles the common cases: `speed = length / duration` (both factors non-negative  $\rightarrow$  non-negative), `mass_density = mass / volume` (both non-negative  $\rightarrow$  non-negative), and `pow<2>(electric_current)` (`electric_current` is possibly-negative, but an even power of a real scalar is always non-negative). However, the rule fails — silently and incorrectly — on a significant class of ISQ quantities:

**Transcendental factors.** Reactive power is defined as  $Q = U_{\text{rms}} \cdot I_{\text{rms}} \cdot \sin \phi$ . In this context  $U_{\text{rms}}$  and  $I_{\text{rms}}$  are RMS magnitudes, which are non-negative by construction, so a library that models them as separate `non_negative` specs would see only non-negative dimensional factors. The rule would therefore infer `non_negative` for `reactive_power`. But  $\sin \phi \in [-1, 1]$ , so reactive power is signed (positive for inductive loads, negative for capacitive). The inferred tag would be a false guarantee: the  $\sin \phi$  factor is invisible to structural dimensional analysis.

**Explicit negation in a definition.** The Massieu function ( $J = -A/T$ ) and Planck function ( $Y = -G/T$ ) are defined by negating a thermodynamic potential before dividing by temperature. The minus sign is not part of the dimensional formula; the rule sees only the magnitudes and cannot detect it.

**Subtraction.** Helmholtz energy ( $A = U - T \cdot S$ ) and Gibbs energy ( $G = H - T \cdot S$ ) are defined by subtraction. Both share the dimension of energy with `kinetic_energy` — which is `non_negative` — yet both are possibly-negative thermodynamic potentials. The dimensional formula encodes only that the result has units of energy, not whether it is bounded below by zero.

In summary: any structural inference rule operating on dimensional formulas alone is blind to the sign-determining features — scalar coefficients, explicit negation, additive structure — that distinguish a signed from a non-negative named spec. The annotation burden would simply shift from `non_negative` to a more complex scheme with no clear benefit and a new class of silent false positives. The authors therefore concluded that **explicit annotation is the only correct approach for named spec definitions**.

#### § 4.4.2 Inference at call-site arithmetic

Having named spec definitions correctly annotated, one might hope to propagate the flag through `derived_quantity_spec` at the call site — so that `m / V` automatically carries `non_negative` when

both `m` and `V` are `non_negative`. This fails too, but for a different reason: raw C++ scalar values are invisible to the type system.

A user can multiply any quantity by a raw `double` or `int` at any point in an expression chain. That scalar carries no quantity spec and no sign information:

```
quantity Q      = U * I * std::sin(phi); // sin(phi) ∈ [-1, 1]: invisible to type
               system
quantity rho2    = -1.0 * (m / V);        // negative literal: invisible to type
               system
quantity E       = efficiency * power;    // efficiency: a raw double, possibly > 1
               or < 0
```

Any inference rule applied to the named-spec factors in these expressions would have to ignore the raw scalar multipliers entirely — and would therefore produce a false `non_negative` guarantee. There is no viable workaround: `dimensionless` is an identity element that is optimized away before any non-negativity inference could observe it, so wrapping scalars as `quantity<dimensionless>` does not help as well.

In summary: automatic inference is unsafe in both contexts. For named spec definitions it produces false positives on specs whose physical sign domain cannot be read from their dimensional formula. For call-site arithmetic it produces false positives whenever a raw scalar is involved in the expression chain. The only correct model is the one adopted here: **no inference at all; the `non_negative` flag is absent from `derived_quantity_spec` and must be explicitly stated on every named spec that carries it.**

## § 4.5 Runtime enforcement on quantity point

The compile-time tag is already implemented as an experimental feature in [\[mp-units\]](#) and it drives an existing **runtime** enforcement on quantity point. When a point is anchored at the *natural* origin of a non-negative spec, the library automatically attaches the `check_non_negative` bounds policy through conditional inheritance. Every construction or mutation of such a point is then checked without any user code:

```
quantity dist = point<distance_traveled[m]>(5.0); // OK, ≥ 0
quantity bad  = point<distance_traveled[m]>(-1.0); // contract violation
```

The default policy can be overridden by defining a custom origin with a different bounds object — for instance, `clamp_non_negative{}` to silently clamp floating-point rounding noise:

```
inline constexpr struct clamped_length_origin : absolute_point_origin<isq::length,
    clamp_non_negative{}> {} clamped_length_origin;
```

Two practical caveats apply:

- The auto-attachment is keyed on `natural_point_origin<QS>`. A point with a different origin is unconstrained by default, even when `QS` is `non_negative`; bounds must be attached explicitly to that origin if needed.
- A point produced from a bare SI unit (`point<m>(5.0)`) deduces its origin as `natural_point_origin<kind_of<isq::length>>`. Because `kind_of<...>` is never `non_negative`, such a point is **not** auto-bounded. To get auto-attachment, use a named spec: `point<distance_traveled[m]>(5.0)`.

User-defined origins — `absolute_point_origin` and `relative_point_origin` — do **not** inherit `check_non_negative` automatically. They start without a bounds policy, and the user supplies one explicitly as a non-type template parameter:

```
// Explicit check — same semantics as natural_point_origin, but user-declared:
inline constexpr struct zero_mass : absolute_point_origin<mass, check_non_negative{}> {} zero_mass;

// Clamping — silently clamp rounding noise rather than firing a contract:
inline constexpr struct zero_mass_clamped : absolute_point_origin<mass, clamp_non_negative{}> {} zero_mass_clamped;
```

This separation is intentional: `natural_point_origin` has an unambiguous physical meaning (the true zero of the quantity), so the library can safely enforce non-negativity there. User-defined origins may represent calibrated offsets, tare baselines, or other contexts where the valid range must be specified explicitly — or may intentionally permit negative values (e.g., a tare origin where net measurements can go negative). The constraint also cannot be bypassed by converting a non-negative absolute quantity to a point and back.

The complete bounds policy design — clamping, wrapping, reflecting, two-sided checks, and hierarchical bounds validation — is described in [Range-validated quantity points](#).

## § 4.6 Why a compile-time tag is not enough on its own

The compile-time tag plus the point-side runtime check are already a substantial safety improvement, and they cover everything the library can do as long as it has only two value abstractions (points and deltas). They do not, however, change the **arithmetic abstraction** of an unanchored value:

- A quantity `<isq::mass[kg]>` in [\[P3045R7\]](#) is a *delta* — it acts like a vector, can be negated freely, and the type carries no concept of a physically meaningful zero anchor.
- Two such quantities can be added, subtracted, multiplied, or divided without restriction; the type system has no way to express that the *physical* domain is the half-line  $[0, +\infty)$  for a value that is not a quantity point.
- The `non_negative` tag exists, but apart from the point-side auto-bound described above there is nowhere in the unanchored case for the library to *use* it.

The next chapter introduces the **absolute quantity** abstraction. Its role here is twofold: it gives the `non_negative` tag a value-level type whose construction can be checked (a plain quantity delta has no such boundary today), and it separates *quantities anchored at true zero* from *signed deltas* in the type system so that arithmetic on non-negative quantities cannot silently produce a signed result.

## § 5 Absolute quantities

### § 5.1 Overview

We propose introducing a third quantity abstraction — the **absolute quantity** — that sits between deltas and points in the type hierarchy:

| Feature                   | Point                               | Absolute              | Delta |
|---------------------------|-------------------------------------|-----------------------|-------|
| Multiplication / Division | ✗                                   | ✓                     | ✓     |
| Addition (A + A)          | ✗                                   | ✓                     | ✓     |
| Subtraction (A – A)       | ✓                                   | ✓                     | ✓     |
| May be non-negative       | ✓                                   | ✓                     | ✗     |
| Physical origin           | Point origin (explicit or implicit) | Anchored at true zero | N/A   |
| Can use offset units      | ✓                                   | ✗                     | ✓     |
| Text output               | ✗                                   | ✓                     | ✓     |

Absolute quantities are anchored at a physically meaningful true zero and may be non-negative. They participate fully in arithmetic: addition, subtraction, multiplication, and division are all well-defined. They can be printed. They cannot use offset units (because they must remain anchored to absolute zero, not a conventional origin).

[ *Note*: An any-sign absolute and a delta share the same mathematical structure: both are vector spaces with a distinguished zero element. The type distinction exists to encode physical interpretation — an absolute is a ratio-scale measurement anchored at true physical zero; a delta is a displacement with no physical zero anchor. The non\_negative absolute is where the two genuinely diverge in mathematical structure: it forms a convex cone  $[0, +\infty)$ , which is closed under addition but not under negation, and is therefore not a vector space. — *end note* ]

This three-way split mirrors the hierarchy found in measurement theory and physics [\[Rosten2025\]](#):

| Concept                              | Measurement Scale | Mathematical Structure                                                       | Physical Meaning                                                                           |
|--------------------------------------|-------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Point</b>                         | Interval Scale    | Affine Space                                                                 | A reading relative to an explicit origin (e.g., 20 °C, altitude above sea level)           |
| <b>Absolute</b> (any sign permitted) | Ratio Scale       | Vector Space (real line); zero = true physical zero                          | A ratio-scale quantity whose sign is unconstrained (e.g., electric charge, reactive power) |
| <b>Absolute</b> (non_negative)       | Ratio Scale       | Absolute Convex Space — convex cone $[0, +\infty)$ with distinguished origin | A non-negative amount; true zero is physically meaningful                                  |
| <b>Delta</b>                         | —                 | Vector Space                                                                 | A change, displacement, or interval                                                        |

Points on a scale with a non-trivial lower bound but *no* distinguished absolute origin occupy a fourth category in [\[Rosten2025\]](#): the **convex space** — a half-line whose lower bound is fixed but not zero. In the library, the Celsius scale is the canonical example: `ice_point` is a `relative_point_origin` offset from the natural Kelvin zero by 273.15 K, so the lower bound of 0 K on the Kelvin absolute space translates automatically to –273.15 °C on the Celsius scale. No explicit

bounds policy is required on `ice_point` — the translation itself enforces the physical lower bound. For other convex-space domains (e.g., a pressure sensor range that starts above zero), an explicit `check_in_range` or `clamp_to_range` policy on the origin provides the equivalent enforcement.

## § 5.2 Syntax

With absolute quantities as the default, the API becomes:

```
quantity q1 = 20 * kg;           // absolute quantity (measured from true zero)
quantity q2 = delta<kg>(20);    // delta quantity (signed difference)
quantity q3 = point<kg>(20);    // point quantity (relative to an origin)
```

producing:

```
static_assert(std::is_same_v<decltype(q1), quantity<kg>>);
static_assert(std::is_same_v<decltype(q2), quantity<delta<kg>>>);
static_assert(std::is_same_v<decltype(q3), quantity<point<kg>>>);
```

[ *Note*: `quantity<point<R, P0>>` replaces the `quantity_point<R, P0>` class template from [P3045R7] — e.g., `quantity_point<isq::altitude[m], mean_sea_level>` becomes `quantity<point<isq::altitude[m], mean_sea_level>>`. The full mapping and the motivation for encoding the “point” context in the reference type are in § [Why `quantity<point<R>>` replaces `quantity\_point<R>`](#). — *end note* ]

This mirrors the way physicists write equations:  $m = 10 \text{ kg}$  denotes an absolute mass, with an explicit  $\Delta$  only when a difference is intended. It is important to note that making quantity default to absolute is a **breaking change** relative to [P3045R7], where quantity is a delta.

## § 5.3 Absolute quantities are not points

A common reaction to absolute quantities is to ask whether the abstraction is really necessary — could we not simply give points the ability to be added and multiplied, and call it done?

The answer is **no**, and the reason lies in the distinction between **interval scales** and **ratio scales** in measurement theory.

An affine point lives on an *interval scale*:

- $\text{point} \pm \text{delta} \rightarrow \text{point}$  (translation along the scale)
- $\text{point} - \text{point} \rightarrow \text{delta}$  (difference between two locations)
- $\text{point} + \text{point}$  is *ill-formed* (adding two locations has no geometric meaning)
- $\text{point} \times \text{point}$  is *ill-formed* (no ratio-scale operations)
- $\text{point} / \text{point}$  is *ill-formed* (no meaningful ratio between two locations)

Timestamps, calendar dates, and geographic positions are interval-scale: it makes no sense to add Tuesday to Friday, or to ask what fraction Berlin is of Paris.

An absolute quantity lives on a *ratio scale*:

- All point operations remain well-defined.
- **abs** + **abs** → **abs** — combining two masses produces a heavier mass.

- **abs × abs → abs** — power × duration = energy is a meaningful product of two absolutes.
- **abs / abs → dimensionless absolute** — mass / volume = density, T\_cold / T\_hot in Carnot efficiency. Ratios between absolute quantities are physically meaningful precisely because they share a common true zero.

So an absolute quantity is **not** “a point with arithmetic added on top”. The defining property of a ratio scale — and the entire reason the third abstraction exists — is that two values are not merely *located* on the scale, but related to a *shared, physically meaningful origin*. That shared origin is what makes their sum, product, and ratio physically meaningful. A point on an arbitrary origin (the floor of a building, the GPS epoch, the freezing point of water) carries no such meaning.

## § 5.4 Scalar and vector quantities

Absolute quantities are always **scalar**: they are ratio-scale magnitudes anchored at a physically meaningful true zero, and the concept of direction — which would require an orientation relative to a reference frame, not just a reference zero — has no place in the model. Vector quantities live in a vector space: they have a mathematical zero (the zero vector, representing no displacement), but not a ratio-scale physical anchor; they represent displacements from a reference and therefore always fall in the **delta** category. Points carry a concrete origin and may be scalar or vector depending on the space they inhabit.

| Category        | Scalar                            | Vector                                               |
|-----------------|-----------------------------------|------------------------------------------------------|
| <b>Point</b>    | point<time>,<br>point<altitude>   | point<position_vector>,<br>point<velocity>           |
| <b>Delta</b>    | delta<duration>,<br>delta<height> | displacement, velocity (no wrapper needed)           |
| <b>Absolute</b> | duration, height, mass            | (none — use <i>norm()</i> to obtain scalar absolute) |

Named vector quantities such as displacement and velocity are displacements in a vector space; their zero is the zero vector (no displacement), not a ratio-scale physical anchor, and they belong to the delta category by construction. To obtain a scalar absolute from a vector quantity, take the norm: `norm(velocity) → speed`.

## § 5.5 Conversions between abstractions

| From\To         | Point                                                      | Absolute                                                                                                                         | Delta                                             |
|-----------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| <b>Point</b>    | Identity;<br>point_for(origin) to<br>change representation | .absolute() (if not offset<br>unit and the origin is<br>natural_point_origin);<br>otherwise (point -<br>origin_or_qp).absolute() | point - origin_or_qp;<br>delta_from(origin_or_qp) |
| <b>Absolute</b> | Explicit ctor<br>using<br>natural_point_origin             | Identity                                                                                                                         | Implicit construction;<br>.delta()                |
| <b>Delta</b>    | origin + delta → point                                     | .absolute() (precondition:<br>non-negative);                                                                                     | Identity                                          |

| From\To | Point | Absolute                                                                             | Delta |
|---------|-------|--------------------------------------------------------------------------------------|-------|
|         |       | always safe: <code>abs()</code> , <code>norm()</code> ,<br>or <code>modulus()</code> |       |

[ *Note*: `delta_from(origin_or_qp)` replaces [\[P3045R7\]](#)'s `delta_from()`. — *end note* ]

[ *Note*: The **absolute** → **delta implicit conversion** is a deliberate design choice. It makes common assignment and initialization convenient — `quantity<delta<m>> x = 40 * m;` works without any explicit cast — and mirrors how absolute magnitudes are routinely used as signed differences in practice (e.g. passing a mass to a function that takes a mass change). The consequence is that code can silently pass an absolute where a delta is expected without any explicit marker at the call site, which is one of the sources of the residual argument-swapping risk shown in [Mass balance revisited](#). SG6 may wish to consider whether this conversion should require an explicit `.delta()` call or explicit construction (analogous to the explicit constructor used in the point case). — *end note* ]

The following examples demonstrate the explicit and implicit casts between all three abstractions:

```
//      —————      From      Absolute
quantity<isq::mass[kg]> total{5 * kg};

// Absolute → Delta: implicit (no notation at call site) or explicit .delta()
quantity<delta<isq::mass[kg]>> d1 = total;           // implicit
quantity<delta<isq::mass[kg]>> d2 = total.delta();    // explicit

// Absolute → Point: explicit construction only (attaches natural_point_origin)
quantity<point<isq::mass[kg]>> p{total};             // explicit ctor

//      —————      From      Delta
quantity<delta<isq::mass[kg]>> change{3 * kg};

// Delta → Absolute: explicit .absolute(); fires contract assertion if change < 0
quantity<isq::mass[kg]> amount = change.absolute();
quantity<isq::mass[kg]> safe   = abs(change);        // always safe: |change|

// Delta → Point: origin + delta
quantity<point<isq::length[m]>> pos =
    natural_point_origin<isq::length> + delta<isq::length[m]>(3);

//      —————      From      Point
quantity<point<isq::thermodynamic_temperature[K]>> boiling{373.15 * K};

// Point → Delta: subtract a reference origin or another point
quantity<delta<isq::thermodynamic_temperature[K]>> above_zero =
    boiling.delta_from(natural_point_origin<isq::thermodynamic_temperature>);

// Point → Absolute: only valid when the point's origin is natural_point_origin
quantity<isq::thermodynamic_temperature[K]> t_abs = boiling.absolute();
```

One might ask whether `delta` → `absolute` should be implicit when the target has no non-negative constraint — after all, no check fires, so the conversion is always safe. The rule is kept **uniformly explicit** for two reasons. First, the category distinction between absolute and delta is not merely about safety: it reflects whether a value is a magnitude on a ratio scale or a signed displace-



ment. That distinction is meaningful even when both signs are permitted, and the explicit `.absolute()` call documents the intent at the call site. Second, a context-sensitive rule (“implicit if possibly-negative, explicit if non\_negative”) would be harder to teach and harder to specify, while buying very little in practice — the call site annotation is cheap. SG6 feedback on this point is welcome.

A deeper tension exists for non\_negative absolutes: the paper argues at length in [Non-negative quantities lack a mathematical home](#) that scattered runtime preconditions are the *symptom* of a missing type-system abstraction. Yet `.absolute()` on a non\_negative target reintroduces a precondition at exactly the most fundamental conversion boundary in the new model. This is intentional but requires acknowledgment. The resolution is that preconditions have not been *removed* — they have been *localized*: instead of appearing at every function that consumes a potentially-negative value, one check fires at the single point where the value enters the non-negative domain. Downstream code that operates on a non\_negative absolute quantity can omit all checks. For code that cannot establish the precondition statically and must handle the violation programmatically, a companion `.try_absolute()` could provide the type-safe alternative without relying on contract semantics. This is a **potential extension**, not a firm proposal — it is raised here to make the design tension explicit and to invite SG6 feedback:

```
// Precondition-checked – fires a contract assertion if delta < 0:
quantity<isq::mass[kg]> m = delta_mass.absolute();

// Potential type-safe alternative – no contract, caller handles the error case:
auto result = delta_mass.try_absolute(); // returns std::expected<quantity, E>
if (result) { /* use *result */ } else { /* handle underflow */ }
```

If SG6 finds the `try_absolute()` direction appealing, the return type `std::expected<quantity, E>` would be used, and the error type `E` — candidates include a standard error code type, a dedicated `quantity_error` type, or a unit type for pure success/failure signalling — would need to be agreed upon separately.

## § 5.6 Arithmetic semantics

All arithmetic category rules follow a single principle — the **zero-anchor principle**: an operation preserves the absolute category whenever the result remains anchored at the same physically meaningful true zero as the operands. The full rationale — including why `Absolute ± Delta → Absolute` was chosen and how it resolves the consistency concern between `Absolute ± Delta` and `Absolute × Scalar` — is given in [Design rationale: the zero-anchor principle](#).

The following tables specify the result types of arithmetic operations between the three abstractions.

### § 5.6.1 Addition

| Lhs \ Rhs | Point | Absolute | Delta    |
|-----------|-------|----------|----------|
| Point     | ✗     | Point    | Point    |
| Absolute  | Point | Absolute | Absolute |
| Delta     | Point | Absolute | Delta    |

Key rules:

- Adding two absolute quantities yields an **absolute** — both are anchored at true zero, so their sum is also anchored at true zero (e.g., combining two masses gives a total mass).
- Adding a delta to an absolute yields an **absolute** — shifting a zero-anchored quantity by a signed change preserves the zero-anchor. For non\_negative specs, a runtime check fires if the result would be negative.
- Adding a delta to a delta yields a **delta** — combining two displacements gives a displacement.
- Adding anything to a point yields a **point** (translation along the scale).

```
quantity tank    = isq::mass(500 * kg);           // quantity<isq::mass[kg]>: check: ≥ 0
quantity refuel  = delta<isq::mass[kg]>(200);      // quantity<delta<isq::mass[kg]>>: +200 kg (signed change)
quantity loaded  = tank + refuel;                  // quantity<isq::mass[kg]>: 700 kg (check: ≥ 0) ✓
tank += refuel;                                     // same: tank is now 700 kg (check: ≥ 0) ✓

// To get a signed result instead, explicitly demote before the operation:
quantity balance = tank.delta() + refuel;          // quantity<delta<isq::mass[kg]>>: signed, no non_negative check
```

## § 5.6.2 Subtraction

| Lhs \ Rhs | Point | Absolute | Delta    |
|-----------|-------|----------|----------|
| Point     | Delta | Point    | Point    |
| Absolute  | ✗     | Delta    | Absolute |
| Delta     | ✗     | Delta    | Delta    |

Key rules:

- Subtracting two absolute quantities yields a **delta** — the difference between two zero-anchored values is a signed displacement, not a zero-anchored magnitude. This is the same rule as point - point → delta in affine spaces.
- Subtracting a delta from an absolute yields an **absolute** — removing a signed change from a zero-anchored value preserves the zero-anchor. For non\_negative specs, a runtime check fires if the result would be negative.
- Subtracting an absolute from a delta yields a **delta** — the delta has no zero-anchor to preserve, so the result inherits the delta character regardless of the subtrahend.
- Subtracting an absolute from a point yields a **point** (translation by the negated absolute).

```
quantity before = isq::mass(100 * kg); // quantity<isq::mass[kg]>: check: ≥ 0
quantity after  = isq::mass(80 * kg);  // quantity<isq::mass[kg]>: check: ≥ 0
quantity lost   = before - after;       // quantity<delta<isq::mass[kg]>>: +20 kg (a displacement)

quantity usage   = delta<isq::mass[kg]>(20); // quantity<delta<isq::mass[kg]>>: a signed change
quantity remaining = before - usage;         // quantity<isq::mass[kg]>: 80 kg (check: ≥ 0) ✓
before -= usage;                               // same: before is now 80 kg (check: ≥ 0) ✓
```

```
// To get a signed balance without triggering a check, demote first:
quantity deficit = before.delta() - usage;    // quantity<delta<isq::mass[kg]>>:
        signed, no non_negative check
```

If an absolute result is needed from an absolute – absolute subtraction (e.g., when you know the difference is non-negative), use an explicit conversion:

```
quantity transferred = (before - after).absolute(); // quantity<isq::mass[kg]>:
        check: 100 - 80 ≥ 0 ✓
```

### § 5.6.3 Multiplication and division

| Operation                            | Result   | Physical Meaning                                                                     |
|--------------------------------------|----------|--------------------------------------------------------------------------------------|
| Absolute × Absolute                  | Absolute | A product of two absolute quantities (energy = power × time)                         |
| Absolute × Scalar                    | Absolute | Rescaling by a dimensionless factor (2 × mass stays absolute mass)                   |
| Absolute × Delta                     | Delta    | Absolute scaled by a displacement is a displacement (e.g., area × Δheight → Δvolume) |
| Absolute / Absolute                  | Absolute | A physical ratio (efficiency, density, strain)                                       |
| Absolute / Delta                     | Delta    | Rate of an absolute w.r.t. a signed step                                             |
| Delta × Absolute                     | Delta    | (same as Absolute × Delta — multiplication is commutative)                           |
| Delta × Scalar                       | Delta    | Rescaling a signed difference (factor preserves delta category)                      |
| Delta / Absolute                     | Delta    | A change scaled by a fixed absolute reference                                        |
| Delta / Delta                        | Delta    | A rate of change (velocity = displacement / duration)                                |
| <code>abs(delta_real_scalar)</code>  | Absolute | Magnitude of a scalar delta                                                          |
| <code>modulus(complex_scalar)</code> | Absolute | Modulus of a complex number                                                          |
| <code>norm(delta_vector)</code>      | Absolute | Magnitude of a vector delta (speed = norm(velocity))                                 |

“Scalar” in this table means a raw C++ numeric type (`double`, `int`, etc.) treated as a multiplicative factor without an absolute/delta annotation. For **category propagation** the table treats it as neutral: Absolute × Scalar → Absolute. For **non\_negative flag propagation**: multiplying a named spec by a raw scalar retains the named spec — `scalar * height` still has spec `height`, so if `height` is `non_negative` the flag is present and a runtime check fires normally (including when a negative scalar drives the value below zero). A `derived_quantity_spec` arises only when at least two distinct quantity specs are combined and do not simplify to a known named spec; in that case the `non_negative` flag is absent and no check fires. The case of a negative scalar multiplier is discussed in [Multiplication by a negative scalar](#).

### § 5.6.4 Runtime enforcement of the non\_negative flag

The `non_negative` flag is a separate, orthogonal property from the absolute/delta/point category. The absolute category alone does **not** imply `non_negative`. Among the base quantities, `electric_current` is the only one that is possibly-negative (currents flow in both directions); all other

base quantities are non-negative. Derived quantities can also be signed: `reactive_power` is absolute (it has a natural true zero) but is signed — it can be positive (inductive load) or negative (capacitive load). The rules below apply only when the result's named spec carries the `non_negative` flag.

**derived\_quantity\_spec never carries the flag** (see [Why automatic inference cannot be implemented correctly](#)); checks fire only when the result is a named `non_negative` spec — either stored directly in one, or mapped to one by a spec-simplification step inside the library.

**Quantities defined without an explicit quantity\_spec — using only a unit — deduce kind\_of<Q> as their spec.** As described in the [non\\_negative flag](#) chapter, a `kind_of<Q>` is never `non_negative` even when `Q` itself is, because the kind covers the entire quantity tree including signed subtypes. Consequently, *quantity-kind values are never checked* — they are maximally flexible (any value is accepted) but also maximally unsafe (the `non_negative` invariant is not enforced). This is the intentional trade-off for code that works with bare SI units and does not name a specific quantity concept:

```
quantity m = 10 * kg; // deduces kind_of<mass> — no non_negative, no check, any
                    value accepted
quantity m = -10 * kg; // also fine: kind_of<mass> is unsigned-indifferent
```

Checks are activated only when a value is narrowed to a named `non_negative` spec, either at construction or via an explicit conversion. This gives users a clear performance/safety dial: use bare units for maximum speed with no constraints; use a named `quantity_spec` to opt in to enforcement.

Runtime checks appear at exactly three kinds of sites:

1. **Construction** from a value whose spec is *not already* a named `non_negative` spec (e.g., `isq::mass(10 * kg)` — constructing from a bare unit, or `isq::mass(m / V)` — constructing from a `derived_quantity_spec` intermediate). The check is **unconditional** in these cases — it fires regardless of whether the operands were themselves `non-negative` named specs. See [Why checks cannot be further elided](#). **Construction and assignment from a value that is already named non\_negative absolute quantity do not re-check** — the source's type already guarantees non-negativity, so no runtime test is needed regardless of whether the source and target specs are identical (e.g., `height` assigned to `length` does not check, because `height` is itself `non_negative`).
2. **Explicit `.absolute()` conversion** from a delta or a possibly-negative absolute.
3. **`absolute<non_neg> ± delta arithmetic`** — the result stays absolute, but the signed delta may underflow; the check fires at the  $\pm$  operation.

There are no other hidden checks. The following cases illustrate the three patterns:

```
// Case 1: mass_density is non_negative; check fires unconditionally at construc-
           tion ✓
quantity_of<isq::mass_density> auto density(quantity_of<isq::mass> auto m,
                                             quantity_of<isq::volume> auto V)
{
    return isq::mass_density(m / V); // named non_negative spec — check fires at
    construction
}

// Case 2: electric_charge is not non_negative: no check at any site ✓
```

```

quantity<isq::electric_charge[C]> q = I * t;           // no check – spec is
    possibly-negative

// Case 3: quantity kind vs named spec – check fires only on the named spec:
quantity m = 10 * kg;                                   // kind_of<mass>: no
    check, any value ok
quantity m1 = isq::mass(m);                             // runtime check: 10
    ≥ 0
quantity m2 = m1;                                         // no check – rhs is
    already a non-negative absolute
quantity<isq::mass[kg]> transferred = (m1 - m2).absolute(); // runtime check: (m1
    - m2) ≥ 0

// Case 4: abs ± delta → abs fires a check at the operation site (non-negative
    spec):
quantity fuel = isq::mass(500 * kg);                     // runtime check: 500
    ≥ 0
quantity refuel = delta<isq::mass[kg]>(200);             // delta: a signed
    change
quantity burn = delta<isq::mass[kg]>(150);               // delta: a signed
    change
fuel += refuel;                                           // abs + delta → abs;
    check: 700 ≥ 0 ✓
fuel -= burn;                                             // abs - delta → abs;
    check: 550 ≥ 0 ✓

// Escape hatch: demote to delta to suppress the check and allow negative values:
quantity deficit = fuel.delta() - delta<isq::mass[kg]>(600); // delta<mass>: -50
    kg, no check

```

### § 5.6.5 Why checks cannot be further elided

A natural follow-up question is: if all typed factors of a `derived_quantity_spec` are `non-negative`, can the library elide the check at named-spec construction since the result appears provably non-negative?

Consider the density function from Case 1:

```

quantity_of<isq::mass_density> auto density(quantity_of<isq::mass> auto m,
                                             quantity_of<isq::volume> auto V)
{
    return isq::mass_density(m / V);
}

```

Both `m` and `V` have `non-negative` specs, so the intermediate `m / V` has a `derived_quantity_spec<mass, per<volume>>` whose every typed factor is `non-negative`. We could be tempted to state that construction of `quantity_of isq::mass_density` should elide the check. However, the problem is more complex than it seems.

It turns out that routinely correct physics computations produce negative values from exactly such non-negative inputs:

- **Gravitational potential energy:**  $U = -G \cdot m_1 \cdot m_2 / r$ . The gravitational constant  $G$ , both masses, and the distance  $r$  are all `non-negative`. The product  $G \cdot m_1 \cdot m_2 / r$  therefore has all `non-negative` typed factors, but the unary negation yields a quantity of the **same derived spec type** with a negative value. `gravitational_potential_energy` is correctly not `non-negative` and must accept this result.

- **Reactive power:**  $Q_r = U_{\text{rms}} \cdot I_{\text{rms}} \cdot \sin \phi$ . Both  $U_{\text{rms}}$  and  $I_{\text{rms}}$  are non\_negative RMS magnitudes;  $\sin(\phi)$  is a raw scalar in  $[-1, 1]$ . The derived spec's typed factors are all non\_negative, yet the result is signed. `reactive_power` is correctly not non\_negative.

In both cases any check keyed on factor non-negativity would either reject the intermediate computation (if placed at `*/`) or silently accept the negative value when elided at named-spec construction.

The root cause in both examples is the same: **raw C++ scalar values carry no sign information at the type level**. The unary `-` applied to  $G \cdot m_1 \cdot m_2 / r$ , and the  $\sin(\phi)$  scalar multiplier, are invisible to the quantity type system. `derived_quantity_spec` has no way to encode that a negative component was introduced: the type of `-1.0 * (m / V)` is identical to the type of `m / V`. If the type system could distinguish them — for example by producing a `possibly_negative<derived_quantity_spec<...>>` type for scalar multiplications whose sign is unknown — both alternatives would become viable. In the absence of that capability, the **unconditional check at named-spec construction** is the only sound placement.

The accepted trade-off is one branch per non\_negative named-spec construction and assignment. The cost is a single comparison — negligible in the context of any real computation.

## § 5.6.6 Unary negation

**Negation of a possibly-negative absolute** is straightforward: `-abs` yields an absolute of the same quantity specification. No category change occurs; the `non_negative` flag (already absent) remains absent.

**Negation of a non\_negative absolute** is an open question. The result is guaranteed to be non-positive, which violates the `non_negative` invariant. Two choices exist:

1. **Compile-time error** — forbid `-abs` when the spec is `non_negative`. Forces the user to convert to a delta first: `(-abs).delta()`. Maximally safe but potentially verbose in formulas that negate a physically non-negative quantity.
2. **Implicit degradation to delta** — `-abs` yields `delta<Q>` when `Q` is `non_negative`. The sign change signals that the result is no longer a magnitude, and the category correctly propagates: `delta / absolute = delta`, `delta ± delta = delta`, etc.

The concrete dilemma arises whenever a formula negates a quantity whose spec is genuinely `non_negative`, such as kinetic energy:

```
// kinetic_energy is non_negative
quantity<isq::kinetic_energy[J]> Ek = ...;
quantity<isq::thermodynamic_temperature[K]> T = ...;

// Option 1: compile error — -Ek is ill-formed; must write (-Ek.delta()) / T
auto J1 = (-Ek.delta()) / T; // delta<kinetic_energy> / absolute<temperature> =
    delta<...>

// Option 2: -Ek yields delta<kinetic_energy> implicitly
auto J2 = -Ek / T;           // delta<kinetic_energy> / absolute<temperature> =
    delta<...>
```

**SG6 direction sought.** Option 2 is physically honest (a negated magnitude is a signed displacement, not a zero-anchored magnitude) and consistent with affine-space conventions, but silently

changes the return type depending on whether the quantity spec happens to be `non_negative`. In generic code, `auto result = -x;` should either always compile with a consistent return type or fail loudly — a silent category change is harder to reason about in templates than a compile error that forces the caller to be explicit. Option 1 makes the violation visible at the negation site and requires only a minor syntactic change (`-Ek.delta()` instead of `-Ek`). The author’s tentative preference is **Option 1**: preferring an explicit conversion over a silent semantic change in return type.

Straw poll: *Negation of a non\_negative absolute should be ill-formed (Option 1), requiring an explicit `.delta()` conversion, rather than silently yielding a delta (Option 2).*

## § 5.6.7 Multiplication by a negative scalar

A raw C++ scalar (`double`, `int`) carries no `non_negative` flag. The category rule is `Absolute × Scalar → Absolute`: the named spec of the quantity operand is preserved, and a runtime check fires on the result when the spec is `non_negative`.

```
quantity m = isq::mass(5 * kg); // non_negative spec; check: 5 ≥ 0 ✓
quantity m2 = 2.0 * m;          // spec: mass (non_negative); check: 10 ≥ 0 ✓
quantity m3 = -1.0 * m;         // spec: mass (non_negative); check: -5 ≥ 0 ✗ -
                                caught
```

A negative mass violates the `non_negative` invariant of `mass`, so the check at the multiplication site is correct — it reflects a genuine precondition violation, not a false positive. If a signed mass difference is intended, use the delta escape hatch:

```
quantity dm = -1.0 * m.delta(); // or: -m.delta()
                                // delta<mass>: a signed change, no check, -5
                                kg is valid
```

Multiplying by a `quantity<dimensionless[one]>` behaves the same as multiplying by a raw scalar: `dimensionless` is the multiplicative identity and is elided during spec simplification before the `non_negative` check is applied, so the named spec of the other operand is preserved and the check fires normally.

## § 5.7 Design rationale: the zero-anchor principle

The rule **`Absolute ± Delta → Absolute`** is the most visible choice in the arithmetic tables. The initial design of the library proposed the opposite rule — `Absolute ± Delta → Delta` — on the grounds that any such operation *can* produce a negative result, so the type should conservatively demote to the unrestricted delta space. This section derives the correct rule from first principles and explains why that initial reasoning, while intuitive, conflates two orthogonal concerns: the category of a quantity and the sign constraint on its value.

### § 5.7.1 The zero-anchor principle

The key design question for `absolute ± delta` is: what property of the operands determines the result category?

The answer is the **zero-anchor principle**: an operation preserves the absolute category when the result remains anchored at the same physically meaningful true zero. A delta is a signed displacement with no inherent zero anchor. Adding or subtracting a displacement from a zero-anchored quantity does **not** destroy the zero anchor — the result is still measured from the same origin:



- `mass_tank + Δ_refuel` → the total mass is still measured from absolute zero (0 kg).
- `mass_tank - Δ_burnt` → what remains is still a mass anchored at zero.

This is precisely the rule used for points in affine spaces: `point ± delta → point`. The structural parallel is not a coincidence; both rules express “a translation preserves the category relative to its origin”. For points the origin is the affine origin; for absolutes it is the natural true zero. Unlike points, however, absolutes live on a ratio scale and support multiplication, division, and addition between two absolutes — operations that are ill-formed for points. The category of the operands (absolute vs. delta) is what drives the result type, not the affine origin.

### § 5.7.1.1 Rejected alternative: universal demotion to delta

The initial design demoted *all* `absolute ± delta` to `delta`, reasoning that since a `delta` can be negative, the result “might” be negative and the type should conservatively reflect that. This was rejected for three reasons:

1. **Type demotion is the wrong mechanism for value-range violations.** The result of `abs ± delta` may be negative at runtime, but so may `abs × scalar`: a negative scalar drives the result below zero, yet `Absolute × Scalar → Absolute` was never in doubt. The correct response to a possible value-range violation is a runtime contract check, not a compile-time type demotion.
2. **It conflates category with value constraint.** Possibly-negative absolutes like `reactive_power` belong on a ratio scale regardless of their sign. Under universal demotion, `reactive_power + delta<reactive_power>` would yield `delta<reactive_power>`, stripping the quantity type and making it impossible to store the result back into a `reactive_power` variable without an explicit conversion.
3. **It penalizes strongly-typed operands over untyped scalars.** A raw C++ scalar can be negative at runtime, yet `Absolute × Scalar → Absolute` was never questioned. A `delta` is a stricter, more informative type than a raw scalar — yet the rejected rule would demote the result further than the weakly-typed case, with no principled justification.

### § 5.7.2 Consistency with `Absolute × Scalar → Absolute`

Reason 3 above is not merely an aesthetic complaint: it reveals a deeper structural requirement. The rules for `Absolute × Scalar` and `Absolute ± Delta` must be consistent because both operations are “neutral on the zero-anchor” — neither one moves the origin. Under the zero-anchor principle both preserve the absolute category, and the code reads naturally as a consequence:

```
// Both rules are consistent under the zero-anchor principle:
quantity m1 = isq::mass(5 * kg);           // absolute
quantity<isq::mass[kg]> m2 = 2.0 * m1;      // absolute × scalar → absolute: 10 kg
quantity d = delta<isq::mass[kg]>(3);       // delta
quantity<isq::mass[kg]> m3 = m1 + d;        // absolute ± delta → absolute: 8 kg ✓
// Both multiply-by-scalar and add-delta are "neutral on the zero-anchor" → consistent result type
```

### § 5.7.3 Handling underflow: when `delta > absolute`

Under the adopted rule, `abs - delta` retains the absolute type. For `non_negative` specs, a runtime contract check fires when the result would be negative. Consider a fuel burn where the maneuver

exceeds the available fuel:

```
quantity fuel          = isq::mass(500 * kg);           // absolute: fuel present in
    the tank
quantity consumption = delta<isq::mass[kg]>(600); // delta: how much the maneuver
    burns
fuel -= consumption; // contract violation: result -100 kg violates the non_nega-
    tive invariant
```

This is the correct behavior for code that must not proceed with an empty — let alone negative — tank: the contract fires before the impossible state is reached. Two alternatives exist for code that needs to observe or react to the deficit instead.

### § 5.7.3.1 Alternative A: demote to delta before the operation

Use the `.delta()` escape hatch to convert the absolute to a delta before the subtraction. The operation then produces a delta with no invariant, so no check fires and the signed balance is available for inspection:

```
quantity balance = fuel.delta() - consumption; // delta<mass>: -100 kg - deficit,
    no check
if (balance < 0 * kg) {
    quantity shortfall = -balance; // delta<mass>: 100 kg (unsigned deficit
    magnitude)
    // handle shortfall: reduce maneuver, add fuel, abort, ...
} else {
    fuel = balance.absolute(); // re-entry: contract check fires, value ≥ 0 ✓
}
```

The explicit `.delta()` call at the subtraction site is the signal to the reader that this code is deliberately working in the signed world.

### § 5.7.3.2 Alternative B: model consumption as an absolute

Fuel consumption is an amount of mass burned — always non-negative, ratio-scale — so it is physically better modeled as an **absolute** rather than a delta. A delta type would permit a negative value, which for this variable represents a non-physical state (refueling is a separate operation with its own named variable); an absolute enforces the non-negativity invariant at construction. With both operands absolute, the rule `abs - abs → delta` applies: the difference between two fuel levels is naturally a signed balance, and no `.delta()` escape hatch is required:

```
quantity fuel          = isq::mass(500 * kg); // absolute: fuel present in the tank
quantity consumption = isq::mass(600 * kg); // absolute: mass the maneuver burns
    (≥ 0)
quantity balance      = fuel - consumption; // abs - abs → delta<mass>: -100 kg,
    no check
if (balance < delta<isq::mass[kg]>(0)) {
    quantity shortfall = -balance; // delta<mass>: 100 kg (unsigned
    deficit magnitude)
    // handle shortfall: reduce maneuver, add fuel, abort, ...
} else {
    fuel = balance.absolute(); // re-entry: contract check fires,
    value ≥ 0 ✓
}
```

Alternative B is the stronger design: the type of consumption encodes the physical invariant, the `abs - abs → delta` rule delivers the signed balance without any explicit demotion, and a negative

consumption value is caught at the call site rather than silently propagated.

For possibly-negative absolutes (those without `non_negative`), `abs ± delta` never fires a check — the spec already permits negative values.

A separate question — orthogonal to the rule choice — is what should happen in contract-disabled environments. The answer is consistent with the rest of the design: contract semantics are configurable; with checks disabled the program proceeds with the underflowed value as a programmer error, just as it does today for any other contract violation in the library.

For domains that require **always-on** enforcement independent of build mode — safety-critical systems, input validation at system boundaries, financial calculations — the `[mp-units]` reference implementation provides `constrained<T, ErrorPolicy>` as a transparent wrapper around a representation type. It carries an error policy as a compile-time tag and specializes `constraint_violation_handler<Rep>`, the customization point that bounds checks query when deciding how to report a violation. With `constrained<double, throw_policy>` as the representation type, every `non_negative` check unconditionally calls `throw_policy::on_constraint_violation()` and throws `std::domain_error` — regardless of whether contracts are compiled in or out:

```
#include <mp-units/constrained.h>

using safe_double = mp_units::constrained<double, mp_units::throw_policy>;

quantity<isq::mass[kg], safe_double> fuel      = isq::mass(500.0 * kg);
quantity<isq::mass[kg], safe_double> consumption = isq::mass(600.0 * kg);
quantity balance = fuel - consumption; // delta<mass, safe_double>: -100 kg, no
    check
fuel = balance.absolute();              // always throws: -100 ≥ 0 violated
```

This mechanism is **not proposed as part of the current paper**. The `constrained` wrapper and the `constraint_violation_handler` customization point are an extension layer that can be standardized separately, with no changes to the core arithmetic rules or the `non_negative` flag semantics described here. Any implementation that adopts the rules in this paper will be forward-compatible with such an addition.

## § 5.7.4 Algebraic-equivalence asymmetry

An objection raised in private review (Yongwei Wu) observes that the algebraic identity

$$a + b = c \iff a = c - b$$

does not preserve types. If `a`, `b`, `c` are all absolutes, the left-hand side is `absolute + absolute → absolute`, but the right-hand side `c - b` is `absolute - absolute → delta`. The rewritten form `a = c - b` then assigns a `delta` to an `absolute`, which is a type error.

This asymmetry is intrinsic to the three-category model and **cannot be removed** by any rule choice. The rule `absolute - absolute → delta` reflects a physical fact: the *difference* between two zero-anchored magnitudes is a signed displacement, not a new magnitude. For `non_negative` absolutes this is especially clear — a convex cone is closed under addition but not under subtraction by definition — but the asymmetry holds for all absolutes. Even `electric_current - electric_current → delta<electric_current>`: the displacement between two current values is a signed change, not a current level. Removing the asymmetry would require allowing `abs - abs → abs` with a possibly-negative result — abandoning the distinction between `absolute` and `delta` and collapsing the ab-

straction the model provides. The asymmetry is therefore a feature, not a bug: the formal expression of “a displacement is not a magnitude, even when both operands are magnitudes”.

```
// The identity  $a + b = c$  does NOT mean  $c - b$  has the same type as  $a$ :
quantity<isq::mass[kg]> a = 30 * kg;           // absolute
quantity<isq::mass[kg]> b = 70 * kg;           // absolute
quantity<isq::mass[kg]> c = a + b;             // absolute: 100 kg

// quantity<isq::mass[kg]> a2 = c - b;           // ✗ delta<mass> ≠
//   absolute<mass> (type error)
quantity<delta<isq::mass[kg]>> diff = c - b;     // delta<mass>: 30 kg -
//   correct displacement type
quantity<isq::mass[kg]> a2 = diff.absolute(); // explicit re-entry: check
//   fires, value 30 ✓
```

## § 5.8 Temperature revisited

With absolute quantities, the temperature use case from [The temperature trap](#) is resolved naturally — and not merely at the API level. The key change is in how the Kelvin unit is defined.

In [\[P3045R7\]](#), kelvin declared `absolute_zero` as its explicit point origin, which triggered the multiply syntax restriction and prevented `28 * K` from compiling. Under the absolute quantities model, kelvin no longer needs a declared point origin — removing the multiply syntax restriction. `absolute_zero` is retained as a named `relative_point_origin` at 0 K to support scenarios like the `temp - absolute_zero` idiom shown in [The temperature trap](#). `ice_point` is defined as an offset in millikelvin from the natural zero:

```
inline constexpr struct kelvin : named_unit<"K", kind_of<isq::thermodynamic_tem-
    perature>> {} kelvin;

// absolute_zero: kept as a named origin for the (temp - absolute_zero) idiom
inline constexpr struct absolute_zero : relative_point_origin<point<kelvin>(0)>
    {} absolute_zero;

// Celsius chain – offset value 273.150 K unchanged
inline constexpr struct ice_point : relative_point_origin<point<milli<kelvin>>
    (273'150)> {} ice_point;
inline constexpr struct degree_Celsius : named_unit<symbol_text{u8" °C ", "`C"},
    kelvin, ice_point> {} degree_Celsius;
```

With this definition, `28 * K` now creates an absolute thermodynamic temperature directly — no conversion idiom required:

```
quantity T = 28. * K;           // absolute temperature – compiles in new
//   design
```

The ideal gas law can be written directly with absolute temperatures:

```
quantity P = 1. * atm;
quantity V = 1. * L;
quantity n = 1. * mol;
quantity T = 301.15 * K;           // absolute temperature
quantity R_calc = P * V / (n * T); // compiles, correct
```

When working with Celsius input, the conversion path is explicit but straightforward:

```
// auto T = 28. * deg_C;           // still does not compile – offset unit
quantity temp = point<deg_C>(28.); // 28 °C on the Celsius scale
```

```
// quantity T = temp.absolute();           // does not compile – offset unit, no
      natural_point_origin
quantity T = temp.in(K).absolute();         // 301.15 K – absolute temperature
quantity R_calc = P * V / (n * T);         // compiles, correct
```

The `.in(K).absolute()` chain is a type-safety checkpoint: it forces the programmer to acknowledge the shift from an interval-scale point (location on the Celsius scale) to a ratio-scale magnitude (thermodynamic temperature measured from absolute zero). This replaces the [P3045R7] workaround `temp.in(K).delta_from_zero()` with a more direct name that reflects the mathematical intent.

Offset units (°C, °F) remain point-only because they carry an explicit non-zero point origin. A point on the Celsius scale is not an absolute temperature — it must be converted explicitly.

[ *Note*: The point abstraction in this library generalizes beyond pure affine spaces. A pure affine space is unbounded — no origin, no bounds — and is the right model for, e.g., spatial position. The Celsius scale is a **convex space** in the taxonomy of [Rosten2025]: it is bounded below at  $-273.15$  °C (the 0 K boundary expressed in Celsius units) but has no ratio-scale physical anchor of its own. The point type covers both: unbounded interval scales (affine spaces) and bounded scales (convex spaces). What they share is the absence of a ratio-scale anchor — they are locations relative to a chosen reference, not amounts measured from true physical zero. — *end note* ]

This distinction has a precise measurement-theory interpretation. Kelvin quantities live in an **absolute convex space**: the domain is the half-line  $[0, +\infty)$  with a physically distinguished origin (absolute zero), and addition of two Kelvin values is well-defined because both are measured from that common anchor —  $T_1 + T_2$  yields a thermodynamically meaningful result. Celsius points, by contrast, are defined as a translation of the Kelvin scale (via `ice_point`): the lower bound of  $-273.15$  °C is not an independent property of the Celsius origin but a direct consequence of that translation — the same 0 K boundary expressed in Celsius units. Adding two Celsius temperatures is ill-formed for the same reason as any affine point addition: a Celsius point is a location on the scale, not an amount measured from true zero. The `.in(K).absolute()` call is precisely the operation that transports a value from the convex-space (Celsius) domain into the absolute convex (Kelvin) domain by making the true-zero anchor explicit.

This design preserves the type-safety that prevents accidental use of offset unit values in thermodynamic equations while providing a natural syntax for the common case (working in Kelvin from the start).

## § 5.9 Mass balance revisited

With absolute quantities, the mass balance use case from [Mass balance and accumulation](#) gains type-safety through the distinction between absolute and delta:

```
quantity<percent> moisture_loss(quantity<delta<kg>> water_lost, quantity<kg>
    total)
{
    return water_lost / total;
}

quantity total_initial = 100 * kg;           // absolute mass
quantity total_dried   = 80 * kg;           // absolute mass
quantity water_lost    = total_initial - total_dried; // delta<mass>: +20 kg
    (difference)
```

```

quantity loss = moisture_loss(water_lost, total_initial); // Correct

// Attempting to swap arguments:
// auto loss = moisture_loss(total_initial, water_lost);
//   arg1: absolute → delta<kg> param: ✓ compiles – absolute→delta is implicit
//   arg2: delta<kg> → absolute param: ✗ fails – delta→absolute requires explicit
//         .absolute()
// The call fails overall, but only because of the second argument.

// A subtler mistake – two absolutes – also compiles:
// auto loss = moisture_loss(total_initial, total_initial);
//   arg1: absolute → delta<kg> param: ✓ implicit
//   arg2: absolute → absolute param: ✓ identity
// Result: total_initial / total_initial = 1.0 (100%) – wrong answer, no
//        diagnostic

```

The three-way split is a **partial** solution to the argument-swapping problem. It eliminates the most common category of mistakes — passing a delta where an absolute is required — but `moisture_loss(total_initial, total_initial)` still compiles because the implicit `absolute → delta` conversion (see the [conversion note](#)) allows `total_initial` to satisfy the `delta<kg>` first parameter silently. This is not a defect in the split; it is a consequence of a deliberate design decision. If SG6 decides to require an explicit `.delta()` call for that conversion, the subtler mistake would also become a compile error. Until then, the residual argument-swap risk remains in the domain of naming conventions and code review.

## § 5.10 Impact on [\[P3045R7\]](#)

This is a **breaking change** to the initial proposal:

1. `quantity<R>` becomes an absolute quantity instead of a delta.
2. Users who need a delta must write `quantity<delta<R>>`, use the `delta<R>(value)` construction helper, or call `.delta()` on an existing absolute quantity.
3. `quantity_point<R, P0>` will be replaced by `quantity<point<R>, P0>` — motivated independently by the need for context-dependent conversions (see [Affine spaces within quantity hierarchies](#)).
4. `absolute` — `absolute` returns a delta, not an absolute (the same rule as `point` — `point` → `delta`). Existing code that assigns such a result to an absolute variable will fail to compile; the fix is either to change the target type to `delta<R>` or to call `.absolute()` on the result to explicitly re-enter the absolute domain (potentially with a runtime contract check).
5. `qp.quantity_from(origin_or_qp)` is renamed to `qp.delta_from(origin_or_qp)` to reflect that it computes a delta (displacement) rather than an absolute quantity.

However:

- Most existing code that uses `quantity` for absolute amounts (mass, distance, speed) continues to work unchanged — those *are* absolute quantities.
- Code that uses `quantity` for signed differences (temperature changes, mass changes) needs explicit `delta<>` wrappers.



- State-update patterns such as `fuel -= consumption` now compile and work correctly for `non_negative` absolutes, with a runtime contract check protecting the invariant.

## § 5.11 Implementation status

The **absolute quantities** abstraction described in this chapter — the three-way split between points, absolutes, and deltas, along with the accompanying arithmetic semantics — is not yet implemented in [\[mp-units\]](#). The mathematical model is, however, proven: Oliver Rosten’s Sequoia library [\[Sequoia\]](#) independently implements the same three-way structural split with a different API surface, confirming that the theoretical taxonomy is sound and implementable in C++. The two authors are working to reconcile their interface choices in this paper; implementation in [\[mp-units\]](#) will follow favorable SG6 direction, with a target of completing it before the WG21 meeting in Brazil.

SG6 is not asked to approve final wording — only to confirm that the direction is worth pursuing. A favorable signal at this stage allows the implementation to validate the specific interface choices proposed here; the design will be refined by that experience before standardization. The breaking-change timeline constraint is real: these are changes that cannot be retrofitted into [\[P3045R7\]](#) after the initial API is locked, which is why early directional consensus is sought ahead of full implementation.

# § 6 Affine spaces within quantity hierarchies

## § 6.1 Motivation

The ISQ defines several pairs of quantities where one naturally behaves as a “position” and the other as a “displacement”:

- **altitude** (a vertical position above a reference surface) and **height** (vertical displacement),
- **position vector** (location in space) and **displacement** (change in position),
- **time** (the ISQ base quantity, used implicitly as the “position” on the time scale) and **duration** (time interval).

In each pair, subtracting two instances of the “position” quantity should yield the “displacement” quantity; adding a displacement to a position should yield a position; adding two positions is meaningless. These rules are precisely the affine space axioms, but they operate at the level of quantity *specifications*, not at the level of the quantity class template.

Today, the library has no way to express these relationships correctly. The exact behavior depends on which hierarchy strategy is used (see [Affine relationships in quantity hierarchies](#)): Option 3 (aliases) erases the distinction entirely; Options 1 and 2 (parent–child) produce wrong result types silently; Option 5 (distinct kinds) blocks any shared arithmetic. The `point_for<>` proposal targets Option 4 (siblings), which preserves the correct ISQ structure but still yields wrong arithmetic without additional annotations. In Option 4, subtracting two `position_vector` quantities yields a `position_vector` — not a displacement, because the two are siblings and the standard tree-walking rules have no concept of a point/delta pairing:

```
quantity pos1 = isq::position_vector(vector{1, 2} * m);
quantity pos2 = isq::position_vector(vector{2, 3} * m);
quantity q = pos2 - pos1; // q is of type position_vector, not displacement
```



In the sibling layout, converting the result to a displacement requires a `quantity_cast` across branches:

```
quantity displacement = quantity_cast<isq::displacement>(q);
```

Worse, adding two position vectors — a physically meaningless operation — compiles without warning, because the library treats both operands as the same quantity type and produces their common type (`position_vector`) again.

## § 6.2 The `point_for<>` attribute

To express point/delta relationships within the hierarchy, we propose a `point_for<>` attribute on `quantity_spec` definitions:

```
inline constexpr struct displacement : quantity_spec<distance, quantity_character::vector> {} displacement;
inline constexpr struct position_vector : quantity_spec<radial_distance, point_for<displacement>, quantity_character::vector> {} position_vector;
```

The annotation `point_for<displacement>` declares that `position_vector` is a “point-like” quantity whose corresponding “delta” is `displacement`. The library uses this annotation to derive the correct result types for arithmetic involving these specifications.

The same pattern applies to scalar quantity pairs:

```
inline constexpr struct height : quantity_spec<length> {} height;
inline constexpr struct altitude : quantity_spec<length, point_for<height>> {} altitude;
```

## § 6.3 Extended quantity specification arithmetic

With `point_for<>` annotations, addition and subtraction of quantity specifications produce correct result types automatically:

```
static_assert(isq::altitude + isq::height == isq::altitude);
static_assert(isq::height + isq::altitude == isq::altitude);
static_assert(isq::altitude - isq::height == isq::altitude);
static_assert(isq::altitude - isq::altitude == isq::height);

// auto qs1 = isq::altitude + isq::altitude; // Compile-time error
// auto qs2 = isq::height - isq::altitude;    // Compile-time error
```

These rules are the direct analog of the affine space axioms from [Arithmetic semantics](#):

- $point + delta \rightarrow point$
- $point - point \rightarrow delta$
- $point + point \rightarrow \text{ill-formed}$
- $delta - point \rightarrow \text{ill-formed}$

The rules compose naturally with the existing hierarchy. When two sibling quantities from different branches interact, the library walks the tree to their common ancestor — as before:

```
static_assert(isq::height + isq::width == isq::length);
static_assert(isq::height - isq::width == isq::length);
```

## § 6.4 Precise convertibility for point-annotated quantities

The `point_for<>` annotation, combined with the three-way reference wrappers (`point<>`, `delta<>`, and bare `absolute`), enables the library to give precise, unambiguous answers to every convertibility question in a related quantity family. This is a feature: where the old library had a single context-blind `quantity_spec` and could not correctly resolve point/delta relationships, the new model encodes context directly in the reference type and resolves each case at compile time.

### § 6.4.1 Valid reference forms

The complete picture for the altitude/height/length family, across all three wrappers:

| Reference                          | Valid? | Notes                                                                          |
|------------------------------------|--------|--------------------------------------------------------------------------------|
| altitude<br>(absolute)             | ✗      | altitude is <code>point_for&lt;height&gt;</code> — the absolute form is height |
| <code>delta&lt;altitude&gt;</code> | ✗      | the delta of altitude is height, not altitude itself                           |
| <code>point&lt;altitude&gt;</code> | ✓      | altitude as a positional quantity                                              |
| height (absolute)                  | ✓      | vertical displacement as a non-negative absolute                               |
| <code>delta&lt;height&gt;</code>   | ✓      | signed vertical displacement                                                   |
| <code>point&lt;height&gt;</code>   | ✓      | height measured from some explicit origin                                      |
| length (absolute)                  | ✓      | generic length as a non-negative absolute                                      |
| <code>delta&lt;length&gt;</code>   | ✓      | signed length delta                                                            |
| <code>point&lt;length&gt;</code>   | ✓      | length measured from some explicit origin                                      |

### § 6.4.2 Arithmetic and conversion

Arithmetic and conversion between these forms are resolved unambiguously:

| Operation                                                                | Result                             | Notes                                                                                              |
|--------------------------------------------------------------------------|------------------------------------|----------------------------------------------------------------------------------------------------|
| height (absolute) → length<br>(absolute)                                 | length                             | height is a specific kind of length absolute                                                       |
| <code>delta&lt;height&gt;</code> →<br><code>delta&lt;length&gt;</code>   | <code>delta&lt;length&gt;</code>   | height is a specific kind of length delta                                                          |
| <code>point&lt;altitude&gt;</code> →<br><code>point&lt;length&gt;</code> | <code>point&lt;length&gt;</code>   | altitude is a specific kind of length point                                                        |
| <code>point&lt;altitude&gt;</code> →<br><code>point&lt;height&gt;</code> | ✗                                  | position is not convertible to displacement                                                        |
| <code>height ± delta&lt;height&gt;</code>                                | height                             | <code>absolute ± delta = absolute</code><br>( <code>non_negative</code> check fires if result < 0) |
| <code>height + point&lt;altitude&gt;</code>                              | <code>point&lt;altitude&gt;</code> | <code>absolute + point = point</code> (absolute implicitly demoted to delta)                       |
| <code>height - point&lt;altitude&gt;</code>                              | ✗                                  | <code>absolute - point</code> is ill-formed                                                        |
| <code>delta&lt;height&gt; +</code><br><code>point&lt;altitude&gt;</code> | <code>point&lt;altitude&gt;</code> | <code>delta + point = point</code>                                                                 |

| Operation                                                                              | Result                             | Notes                                                                  |
|----------------------------------------------------------------------------------------|------------------------------------|------------------------------------------------------------------------|
| <code>delta&lt;height&gt; – point&lt;altitude&gt;</code>                               | $\times$                           | <code>delta – point</code> is ill-formed                               |
| <code>point&lt;altitude&gt; ± height</code>                                            | <code>point&lt;altitude&gt;</code> | <code>point ± absolute = point</code>                                  |
| <code>point&lt;altitude&gt; ± delta&lt;height&gt;</code>                               | <code>point&lt;altitude&gt;</code> | <code>point ± delta = point</code>                                     |
| <code>point&lt;altitude&gt; – point&lt;altitude&gt;</code>                             | <code>delta&lt;height&gt;</code>   | <code>point – point = delta</code>                                     |
| <code>point&lt;altitude&gt; + point&lt;altitude&gt;</code>                             | $\times$                           | <code>point + point</code> is ill-formed                               |
| common type of <code>point&lt;altitude&gt;</code> and <code>height</code>              | $\times$                           | <code>position</code> and <code>absolute</code> are incommensurate     |
| common type of <code>point&lt;altitude&gt;</code> and <code>delta&lt;height&gt;</code> | $\times$                           | <code>position</code> and <code>displacement</code> are incommensurate |

## § 6.5 Why `quantity<point<R>>` replaces `quantity_point<R>`

Encoding context in the reference type is what enables the library to resolve the convertibility rules above at the class-template level. The current library has two separate class templates — `quantity<R>` for deltas (and absolutes) and `quantity_point<R, P0>` for points — both receiving the bare `R`. The conversion machinery cannot determine whether `R` is being used as a point or a delta, which is precisely why it could not differentiate the cases in the table above.

The solution is to encode the context in the reference itself:

| Current API                                                         | Proposed unification                                                       |
|---------------------------------------------------------------------|----------------------------------------------------------------------------|
| <code>quantity&lt;isq::height[m]&gt;</code>                         | <code>quantity&lt;isq::height[m]&gt;</code>                                |
| <code>quantity_point&lt;isq::height[m]&gt;</code>                   | <code>quantity&lt;point&lt;isq::height[m]&gt;&gt;</code>                   |
| <code>quantity_point&lt;isq::altitude[m]&gt;</code>                 | <code>quantity&lt;point&lt;isq::altitude[m]&gt;&gt;</code>                 |
| <code>quantity_point&lt;isq::altitude[m], mean_sea_level&gt;</code> | <code>quantity&lt;point&lt;isq::altitude[m], mean_sea_level&gt;&gt;</code> |

With `point<>` in the reference:

- `point<isq::altitude>` converts to `point<isq::length>` — `altitude` is a specific kind of length point.
- Bare `isq::altitude` without a `point<>` wrapper is rejected — a `point_for<>` quantity has no absolute form.
- `common_quantity_spec(point<isq::altitude>, isq::height)` is ill-formed — `position` and `displacement` are incommensurate.
- A single `quantity` class template handles all three cases, reducing the core API surface.
- The library aligns with ISO/IEC 80000, which discusses “quantities” without making quantity points a separate concept.

This refactoring eliminates `quantity_point` as a separate class template. Even without the absolute/delta distinction from [Absolute quantities](#), the need to distinguish point and delta contexts when evaluating quantity specification conversions independently requires moving the “point” marker into the reference type.

[ *Note*: The `point_for<>` attribute on `quantity_spec` (which declares affine relationships between quantity specifications) is distinct from the `point_for()` member function on quantity points (which converts a value between origins). Both relate to the “point” concept but operate at different levels: `point_for<height>` tells the library that *altitude* is a point type, while `qp.point_for(mean_sea_level)` converts a point value to a different origin. — *end note* ]

## § 6.6 Implementation status

The `point_for<>` attribute, the associated quantity specification arithmetic, and the `quantity<point<R>>` unification described in this chapter are not yet implemented in [\[mp-units\]](#). Unlike the absolute/delta/point split, this is a novel approach with no prior implementation: it emerged directly from the challenges of representing ISQ position/displacement pairs correctly in [\[mp-units\]](#), and no other library has attempted it. Implementation in [\[mp-units\]](#) will follow SG6’s directional consensus, with the same Brazil-meeting target. Refinements — particularly in the interaction between `point_for<>` annotations and the absolute/delta/point three-way split — are possible from that implementation experience.

## § 6.7 Standardization plan

The two features proposed in this chapter have different standardization urgency.

### § 6.7.1 `quantity<point<R>>` unification

The `quantity<point<R>>` unification must be standardized regardless of whether absolute quantities are adopted. Even without the absolute/delta distinction, correct point/delta convertibility requires encoding context in the reference type: the current `quantity_point<R, P0>` receives a bare `R` and cannot determine at the class-template level whether `R` is being used as a point or a delta. If absolute quantities are also adopted, the same mechanism additionally distinguishes the third (absolute) wrapper — all three contexts are then handled uniformly by a single class template.

### § 6.7.2 `point_for<>` attribute

The `point_for<>` attribute is a **pure addition** and can be standardized in a later revision. No existing interface breaks if it is absent — quantity operations simply fall back to the standard tree-walking rules, as they do today. It becomes essential only when the system of quantities shipped with the library includes quantities annotated as point-like, such as `altitude`, `position_vector`, and any quantity derived from them (e.g., `moment_of_force` depends on `position_vector`). If those quantities are not part of the initial standard library release, `point_for<>` can follow in a subsequent revision without any breaking change.

## § 7 Range-validated quantity points

### § 7.1 Overview

Range-validated quantity points attach overflow policies directly to point origins as an additional NTTP, analogous to the way `quantity_spec` accepts extra parameters such as `non_negative`. Bounds are enforced during construction, unit conversion, arithmetic, and origin conversion.

This feature is a **pure addition** — it does not affect the existing quantity or `quantity_point` interfaces and can be added independently of the absolute quantities proposal.

## § 7.2 Design rationale

### § 7.2.1 Constraints belong to the measurement space, not to enforcement parameters

A physical domain constraint is a property of the *measurement space*, not of the *enforcement mechanism*. The non-negativity of mass is a physical law; whether a violation triggers an assertion, clamps the value to zero, or records it for later reporting is a separate, orthogonal concern handled by the contract violation mechanism (or a custom representation type). Embedding the constraint as a quantity *type parameter* would conflate the mathematical structure with the enforcement policy: it would make `quantity<mass[kg], check_non_negative>` and `quantity<mass[kg]>` incompatible types even though both hold the same kind of value, making generic code painful to write.

The design handles the two abstractions differently:

- For **absolute quantities**, non-negativity is expressed as an attribute of the quantity spec itself (the `non_negative` specification). Enforcement is a precondition check at the single boundary where a delta scalar converts to an absolute; downstream arithmetic on an already-validated absolute requires no further checks.
- For **point quantities**, bounds are attached to the *origin* rather than the quantity type. Different origins for the same quantity can carry different bounds (e.g., AGL vs. MSL altitude), and converting between origins validates against the target's bounds automatically.

[ *Note*: A constrained<Rep, Policy> representation type — where Policy governs the enforcement strategy (clamp, throw, report) — is a natural complement at the representation layer and would produce distinct types intentionally, since distinct enforcement semantics warrant distinct types. That concern is orthogonal to the physical-domain bounds modeled here. — *end note* ]

Beyond this mathematical argument, the design choice also follows from the physics: the valid range of a point depends on *which reference frame you are measuring from*, not on the abstract quantity being measured.

For example, altitude measured from mean sea level (MSL) might have a range [−100 m, +10,000 m] to accommodate locations below sea level and high-altitude flight. But altitude measured from ground level (AGL) for a drone might be constrained to [0 m, +120 m] by regulation. Both are altitudes, measured in metres, but the valid ranges differ because the *origins* differ.

Placing bounds on the origin makes this explicit:

- Different origins for the same quantity can have different bounds
- Converting between origins (`point_for()`) automatically validates against the target origin's bounds

- The same quantity point type can be constrained or unconstrained depending on which origin it uses

## § 7.2.2 Why not model bounded variants as distinct quantities in the hierarchy?

One might be tempted to instead introduce `altitude_msl` and `altitude_agl` as sibling quantity types in the quantity hierarchy and attach bounds there. This approach appears attractive at first but breaks down on several fronts:

1. **Hierarchy explosion.** Altitude alone can be measured from MSL, AGL, WGS-84 ellipsoid, geoid, flight level pressure surface, and more. Longitude can be measured from the prime meridian or from arbitrary local meridians. Temperature setpoints can be constrained to HVAC ranges, human comfort bands, industrial process limits, etc. Encoding every such constraint as a distinct quantity type causes an unbounded proliferation of names — most of which express the same underlying quantity merely measured from a different reference.
2. **Conversions require runtime data and break the hierarchy model.** Converting `altitude_agl` to `altitude_msl` requires knowing the ground elevation at the measurement site — a runtime value that depends on geography, not a compile-time property of the quantity type. The quantity hierarchy is a compile-time, unit-independent construct. There is no place in it for runtime offsets. `quantity_cast` and implicit conversions in the hierarchy assume a fixed, known relationship between parent and child; that assumption breaks for reference-frame-dependent conversions. The quantity type system has no mechanism to express “add 100 m when converting.”
3. **Cross-variant arithmetic forces a proliferation of delta types.** Consider the three related quantities `altitude_msl`, `altitude_agl`, and `height` (a vertical displacement). The natural expectation is:

```
altitude_msl + height → altitude_msl
altitude_agl + height → altitude_agl
altitude_msl - altitude_msl → height
altitude_agl - altitude_agl → height
```

For these to work consistently, `height` must be the delta type for both altitude variants. But then `altitude_msl - altitude_agl` would also yield `height` — a numerically wrong result because the ground-elevation offset is silently ignored. Worse, with `height` as a common quantity shared by both variants, `altitude_msl + altitude_agl` would also compile and yield `height` — adding two positions to get a displacement is physically nonsensical, yet the type system would permit it silently. To prevent this, the hierarchy would need to introduce *distinct delta types per variant*: `height_msl` and `height_agl`, so that subtracting mixed variants no longer produces a common type. But then `height_msl` and `height_agl` are physically identical — a 10 m vertical displacement is the same thing regardless of which datum your altimeter is zeroed to — and every algorithm accepting a generic `height` must now be templated over all of them. There is no way to satisfy all four expectations above and prevent cross-variant subtraction without either duplicating the delta type or silently producing wrong results. With origins, this problem never arises: addition of two points is already ill-formed by the affine space rules; subtracting two points from the same origin yields a frame-independent `height`; and subtracting points from *different* origins requires an explicit `point_for()` conversion that applies the known offset — correctly.

4. **Generic code becomes impractical.** A function that computes climb rate accepts any altitude. If altitude variants are distinct types, the function must either be templated on an ever-growing set of altitude types, or accept only one variant, unnecessarily restricting callers. With a single `altitude` quantity and distinct origins, the function continues to accept `quantity_of<isq::altitude>` and remains universally applicable.
5. **Bounds are not a property of the physical quantity.** The ISQ defines `altitude` as “distance measured upward along the local vertical from a reference surface.” The valid range `[-90 m, +10,000 m]` is a property of a particular coordinate system’s coverage area or a regulatory limit — it is not part of the definition of altitude as a physical quantity. Encoding it in the quantity type conflates the physical kind with an operational constraint that can change without the physics changing at all.
6. **Axis inversion cannot be encoded in a quantity type.** A closely related failure mode arises when two measurements differ not by a translational offset but by an **axis inversion** — a sign flip in the measurement direction. The canonical example is `altitude` (positive upward from sea level) and `depth` (positive downward from the ocean surface). Both describe a vertical position; the only difference is the sign convention of the axis. If `depth` is modeled as a sibling `quantity_spec` of `altitude`, all the problems above reappear in acute form:
  - `point<altitude> - point<depth>` (or vice versa) would compile and silently yield a wrong-signed result — the values should be *added* (opposite-direction axes), yet the type system delivers a subtraction as if both axes pointed the same way.
  - The delta type of both `altitude` and `depth` is `height` (a vertical displacement does not depend on which direction the axis points), so the delta-type proliferation from point 3 cannot be resolved by inventing distinct delta types without contradicting physics.
  - There is no mechanism in the quantity type hierarchy to say “negate this value when converting” — that is a property of the coordinate system, not of the physical quantity.

Oliver Rosten raised in LEWGI discussions that `relative_point_origin` cannot express axis-inverting transformations, and that this gap suggests origins are insufficient — and that quantities should bear more of the modeling burden, as his library does. The observation about `relative_point_origin` is correct: it cannot negate. But the response is not to encode axis direction in the quantity hierarchy; it is to provide a richer origin mechanism that *can* express such transforms. That is what `frame_projection` delivers: `ocean_surface` is an `absolute_point_origin<isq::altitude>` connected to `sea_level` by a negating `frame_projection`, and `depth` is simply `point<isq::altitude>` at that origin. No `depth` quantity spec is needed — axis direction belongs in the frame, not in the quantity.

All of these problems dissolve when bounds and axis conventions are placed on origins. The quantity `altitude` remains a single, stable physical concept. The valid range and axis direction become declared properties of the reference frame — exactly where physics says they belong.

### § 7.2.3 Bounds values are deltas, not points

The bounds passed as an NTTP are **delta quantities** (displacements from the origin), not point quantities. This is a deliberate architectural choice that maintains consistency with the affine space model:



```
inline constexpr struct north final
: absolute_point_origin<compass_bearing, wrap_to_range{0 * deg, 360 * deg}> {}
north;
// Bounds are deltas: [0° displacement, 360° displacement)
```

Storing bounds as deltas (rather than absolute points on a scale) ensures:

- The bounds are origin-agnostic — they describe valid *displacements* from whatever origin they are attached to
- Converting between origins does not require translating bound values — the bounds move with the origin
- Half-line bounds (`check_non_negative`, `clamp_non_negative`) have a natural representation:  $[0, \infty)$

## § 7.3 Overflow policies

The library provides four primary overflow policies and two half-line bounds policies:

| Policy                          | Behavior                                                   | Use Case                                       |
|---------------------------------|------------------------------------------------------------|------------------------------------------------|
| <code>check_in_range</code>     | Reports violation via handler                              | Bounds checking with customizable error        |
| <code>clamp_to_range</code>     | Clamps to nearest boundary                                 | Saturating arithmetic, sensor limits           |
| <code>wrap_to_range</code>      | Wraps circularly ( <code>[min, max)</code> )               | Angles, time-of-day, longitude                 |
| <code>reflect_in_range</code>   | Reflects (folds) at boundaries ( <code>[min, max]</code> ) | Geographic elevation angle, bouncing particles |
| <code>check_non_negative</code> | Reports violation if value $< 0$                           | Non-negative quantities (length, mass)         |
| <code>clamp_non_negative</code> | Clamps negative values to zero                             | FP rounding noise in non-negative domains      |

The `check_in_range` and `check_non_negative` policies route violations through the constraint handler — currently `MP_UNITS_EXPECTS`, and ultimately C++ Contracts in the standardized version. Like any contract assertion, they can be disabled in unchecked build modes; in that case, out-of-range values are not caught and behavior is implementation-defined. The clamping policies (`clamp_to_range`, `clamp_non_negative`) have no undefined behavior and no contract dependency — they silently return the nearest in-range value regardless of build mode.

All validation mechanisms proposed in this paper — bounds policies, non-negativity enforcement, and projection-result checking — are required to be usable in constant expressions. A validation failure during constant evaluation renders the enclosing expression non-constant; the failure is therefore ill-formed when the expression is required to be constant (e.g., in a `constexpr` variable initializer or `static_assert`).

More precisely, the checking policies (`check_in_range`, `check_non_negative`) route violations through contract assertions, which follow the contract-mode configuration of the translation unit. When contracts are enabled, a bounds violation in a `constexpr` context is ill-formed and produces a compile-time diagnostic. When contracts are disabled (unchecked build mode), the assertion is a no-

op even during constant evaluation — out-of-bounds `constexpr` values are silently accepted. `static_assert`-based checks (used for hierarchical bounds validation at origin definition time, see [Hierarchical bounds validation](#)) are not affected by contract mode and always fire during constant evaluation. Bounds policies also apply to values returned by `frame_projection` callables when constructing a quantity point at a bounded origin; the constant-evaluation semantics for that case are discussed in § [Projection requirements and failure modes](#).

The first four policies (`check_in_range`, `clamp_to_range`, `wrap_to_range`, `reflect_in_range`) are general-purpose and accept explicit bounds. The last two (`check_non_negative`, `clamp_non_negative`) are specialised half-line bounds that enforce  $[0, \infty)$  — they are particularly useful for quantities that are inherently non-negative but may temporarily violate this constraint due to floating-point rounding errors or user input.

This set of six policies is proposed as the initial, minimal vocabulary, intended to be extended as experience accumulates. The [\[mp-units\]](#) library already prototypes a `constraint_violation_handler<Rep>` customization point and a `constrained<T, ErrorPolicy>` wrapper that together enable always-on, build-mode-independent enforcement as a non-breaking extension on top of these policies; see § [Always-on enforcement with constraint\\_violation\\_handler<Rep>](#).

## § 7.4 Automatic bounds for non-negative quantity specifications

When a quantity specification is marked `non_negative`, the library automatically applies `check_non_negative` to its `natural_point_origin`. User-defined origins do not inherit this policy automatically and must supply one explicitly (see [Runtime enforcement on quantity point](#) for full details):

```
inline constexpr struct mass : quantity_spec<dim_mass, non_negative> {} mass;

// natural_point_origin<mass> implicitly has check_non_negative{} bounds
quantity<point<kg>> m(10 * kg);      // OK – natural_point_origin is the implicit
    anchor
quantity<point<kg>> bad(-5.0 * kg); // ✗ constraint violation reported
```

## § 7.5 Example: angular coordinates

```
inline constexpr struct geo_longitude : quantity_spec<isq::angular_measure> {}
    geo_longitude;

inline constexpr struct prime_meridian
    : absolute_point_origin<geo_longitude, wrap_to_range{-180 * deg, 180 * deg}> {}
    prime_meridian;

template<typename T = double>
using longitude = quantity<point<deg, prime_meridian>, T>;

longitude lon = prime_meridian + 270.0 * deg; // wraps to -90°
```

Each origin is defined in a single declaration — the policy is part of the type, not a separate specialization. This follows the same convention as `quantity_spec<dim_mass, non_negative>`, where extra semantic properties are NTTP parameters of the base class.

## § 7.6 Hierarchical bounds validation

When a `relative_point_origin` defines bounds and its parent origin also has bounds, the library enforces at compile time that the child's bounds fit within the parent's bounds (after translating to the parent's reference frame). This ensures semantic correctness: a room's temperature range should not exceed the building HVAC system's capability.

The validation occurs at the point where the child origin is defined. If the child's bounds do not fit within the parent's bounds, the program is ill-formed — a `static_assert` fires at compile time:

```
inline constexpr struct sea_level : absolute_point_origin<altitude,
    check_in_range{-100 * m, 10'000 * m}> {} sea_level;
inline constexpr struct ground_level : relative_point_origin<sea_level + 100 * m,
    check_in_range{0 * m, 120 * m}> {} ground_level;
// Validates at definition: ground_level bounds [100 m, 220 m] in sea_level frame
// fit within sea_level bounds [-100 m, 10'000 m] ✓
```

This compile-time check prevents **origin hierarchies with semantically inconsistent bounds**. For example, attempting to define a ground-level origin with a range that extends below sea level when the parent sea-level origin does not permit negative altitudes would be caught immediately.

The inheritance rule is: when an origin B is defined relative to another origin A, and both have bounds, the bounds of B (translated into A's reference frame) must be a subset of the bounds of A. This is checked via `static_assert` during constant evaluation of the origin definition.

## § 7.7 Always-on enforcement with `constraint_violation_handler<Rep>`

[ *Note*: `constraint_violation_handler<Rep>` is under consideration for proposal based on committee feedback requesting a mechanism for wide-contract, always-on enforcement. `constrained<T, ErrorPolicy>` is **not proposed for C++29**; it is shown here as an illustrative usage pattern built on top of the customization point. — *end note* ]

Bounds policies on origins route violations through contract preconditions by default, which can be compiled away in release builds. For domains that require guaranteed, build-mode-independent enforcement — safety-critical systems, input validation at system boundaries, financial calculations — the library proposes `constraint_violation_handler<Rep>`: a customization point that any representation type can specialize to register its own violation handler. When a checking policy (`check_in_range`, `check_non_negative`) detects a violation, it queries this handler for the active `Rep`; if a specialization exists, it is called unconditionally in place of the contract assertion, regardless of build mode:

```
// Opt any Rep type into always-on enforcement by specializing the handler
class my_safe_double { /* ... */ };

template<>
struct mp_units::constraint_violation_handler<my_safe_double> {
    static void on_violation(std::string_view msg) {
        throw std::domain_error(std::string(msg));
    }
};
```

The `[mp-units]` library additionally provides `constrained<T, ErrorPolicy>` as a convenience wrapper that automates this pattern — it pre-registers a `constraint_violation_handler` for `constrained<T, ErrorPolicy>` automatically, with built-in policies `throw_policy` and `termi-`

nate\_policy. It is a library convenience, not a new primitive, and could be proposed separately once the customization point is standardized:

```
using safe_double = mp_units::constrained<double, mp_units::throw_policy>;

// Front-end: always-on enforcement via constrained<> Rep
using checked_mass = quantity<point<isq::mass[kg]>, safe_double>;

// Back-end: precondition only – check can be elided for performance
using fast_mass = quantity<point<isq::mass[kg]>, double>;
```

## § 7.8 Interaction with `std::numeric_limits`

When a quantity point has bounds attached to its origin, the `std::numeric_limits` specialization automatically reflects the constrained bounds:

```
quantity lon_min = std::numeric_limits<longitude<>>::min();
// prime_meridian - 180°
```

# § 8 Runtime frame projections

## § 8.1 Motivation

As described in [Limitations of `relative\_point\_origin`](#), the current `relative_point_origin<QP>` design is limited to compile-time constant offsets and structural types. This section proposes `frame_projection` — a customization point that enables runtime-determined and parameter-dependent transformations between quantity point origins.

## § 8.2 Comparison with `relative_point_origin`

Both mechanisms connect pairs of origins and allow transparent conversion via `point_for()`, but they differ fundamentally in where the transformation is computed and what guarantees the library can provide:

| Property                          | <code>relative_point_origin</code>  | <code>frame_projection</code>               |
|-----------------------------------|-------------------------------------|---------------------------------------------|
| Transformation defined            | At compile time (NTTP)              | At runtime (callable specialization)        |
| Representation type requirements  | Must satisfy structural type rules  | No restriction — any callable               |
| Automatic inverse                 | Yes (library derives it)            | No — both directions must be specialized    |
| Compile-time chain discovery      | Yes (library traverses origin tree) | No — only direct pairs are connected        |
| Type-level instance identity      | Encoded in the type                 | Not possible — all instances share one type |
| Zero runtime cost when applicable | Yes                                 | No — calls through the functor at runtime   |
| Suitable for                      | Celsius ↔ Kelvin, epoch shifts      | Geoid undulation, joint kinematics          |

The two mechanisms are complementary. `relative_point_origin` should be preferred whenever the transformation offset is a compile-time constant and the representation type is structural. `frame_projection` fills the remaining cases where a runtime parameter or a non-structural type is unavoidable. Both can coexist in the same program: `point_for()` tries the compile-time path first and falls back to `frame_projection` only when no compile-time path is found.

The main limitation of `frame_projection` compared to `relative_point_origin` is the absence of automatic chain discovery. With `relative_point_origin`, the library can find a path between any two origins connected through the origin tree, even indirectly. With `frame_projection`, only directly specialized pairs are connected — if the user wants to convert from origin A to origin C via B, they must either specialise `frame_projection<A, C>` directly or perform two explicit `point_for()` calls. This is intentional: runtime chains require the user to supply runtime parameters at each step, and composing them automatically would require the library to know in what order and with what arguments to invoke each hop. For the robot arm use case, this means converting joint by joint, explicitly passing each angle, until the world frame is reached.

The motivating use cases (detailed in [Limitations of relative\\_point\\_origin](#)) include:

- Location-dependent altitude conversions (MSL ↔ HAE with geoid undulation)
- Coordinate frame rotations and full affine transformations
- Representation types with private members (decimal, rational, arbitrary-precision)

The robot arm use case from [Limitations of relative\\_point\\_origin](#) is addressed directly: each adjacent joint pair is connected by a `frame_projection` specialization whose callable accepts the current joint angle as a runtime parameter. The user converts joint by joint — one `point_for()` call per hop — until the world frame is reached. At each step the library enforces that the source and destination origins are the correct adjacent pair, so the chain is type-safe even though its transformation parameters are runtime values. The user cannot accidentally skip a joint: skipping would require a `frame_projection` specialization directly between non-adjacent frames, which the user simply would not define.

## § 8.3 Proposed customization point

A new variable-template customization point connects pairs of origins:

```
template<point_origin auto From, point_origin auto To>
inline constexpr /* unspecified */ frame_projection = undefined;
```

When specialized, the specialization must be a callable that converts a quantity point from one origin to another. The library invokes it automatically when `point_for()` determines there is no compile-time translation path.

## § 8.4 Integration with point\_for()

The existing `point_for()` member function is extended to transparently cover both paths and take additional runtime conversion arguments:

```
template<PointOrigin NewPO, typename... Args>
    requires SameAbsolutePointOriginAs<NewPO, absolute_point_origin> ||
             HasFrameProjection<absolute_point_origin, AbsoluteRootOf<NewPO>>
auto point_for(NewPO new_origin, Args&&... args) const
```

```

{
    if constexpr (SameAbsolutePointOriginAs<NewPO, absolute_point_origin>)
        // existing compile-time translation path (unchanged)
        return quantity{*this - new_origin, new_origin}; // result: quantity<point<R,
        new_origin>>
    else {
        // walk up → project → walk down
        auto at_src = point_for(absolute_point_origin);
        constexpr auto abs_tgt = AbsoluteRootOf<NewPO>;
        auto at_tgt = frame_projection<absolute_point_origin, abs_tgt>(at_src,
            std::forward<Args>(args)...);
        if constexpr (is_same_v<NewPO, decltype(abs_tgt)>)
            return at_tgt;
        else
            return at_tgt.point_for(new_origin); // walk down within target
    }
}

```

The caller does not need to know which path is taken — the syntax is identical for compile-time and runtime conversions.

## § 8.5 Bidirectional projections

`frame_projection` requires two explicit specializations for bidirectional conversion. The library does not attempt to derive an inverse from a forward transform — not all projections are invertible, and even when they are, the implementation may differ for numerical reasons.

## § 8.6 Bounds on projected results

When the target origin carries a bounds policy NTTP, `point_for()` applies that policy on the result. Cross-origin bounds consistency checking is not attempted — origins connected by `frame_projection` are logically independent.

## § 8.7 Stateless example: altitude ↔ depth

`depth` is not a separate `quantity_spec` — it is altitude measured in the downward direction (axis-inverted). Both `sea_level` and `ocean_surface` are `absolute_point_origin<isq::altitude>`; the `frame_projection` between them negates the value. Both directions must be specialized:

```

inline constexpr struct sea_level : absolute_point_origin<isq::altitude> {}
    sea_level;
// axis inverted: positive = downward
inline constexpr struct ocean_surface : absolute_point_origin<isq::altitude> {}
    ocean_surface;

template<>
inline constexpr auto frame_projection<sea_level, ocean_surface> =
    [](quantity_point_of<isq::altitude> auto qp) { return ocean_surface -
        qp.delta_from(sea_level); };

template<>
inline constexpr auto frame_projection<ocean_surface, sea_level> =
    [](quantity_point_of<isq::altitude> auto qp) { return sea_level -
        qp.delta_from(ocean_surface); };

// Usage:

```

```

quantity alt = sea_level + (-100. * m);    // 100 m below sea level (altitude =
    -100 m)
quantity d    = alt.point_for(ocean_surface); // depth = 100 m (positive downward)
// d.delta_from(ocean_surface) == 100 * m

quantity d2    = ocean_surface + 100. * m;    // 100 m depth (positive downward)
quantity alt2   = d2.point_for(sea_level);    // altitude = -100 m (below sea
    level)
// alt2.delta_from(sea_level) == -100 * m

```

## § 8.8 Stateless example: geometric azimuth ↔ bearing

The bearing/azimuth case from [Scale, translation, and rotation](#) is also stateless — the sign-inverting formula  $\text{bearing} = 90^\circ - \text{azimuth}$  is a pure compile-time mathematical relationship. The inverse formula  $\text{azimuth} = 90^\circ - \text{bearing}$  has the same structure:

```

template<>
inline constexpr auto frame_projection<east, north_cw> =
    [] (quantity_point_of<isq::angular_measure> auto qp) { return north_cw + (90 *
        deg - qp.delta_from(east)); };

template<>
inline constexpr auto frame_projection<north_cw, east> =
    [] (quantity_point_of<isq::angular_measure> auto qp) { return east + (90 * deg
        - qp.delta_from(north_cw)); };

// Same call sites as before — now b compiles too:
navigate(h);    // ✓ as before — same type
navigate(az);   // ✓ as before — compile-time chain north_ccw → east
navigate(b);    // ✓ now compiles — frame_projection<north_cw, east> bridges the
    gap

```

The library resolves `navigate(b)` by walking bearing up to its absolute root (`north_cw`), applying `frame_projection<north_cw, east>` to reach east, then walking down the `relative_point_origin` chain to `north_ccw` — all transparently, without any explicit conversion at the call site.

## § 8.9 Runtime arguments: camera calibration

For runtime-determined transforms where the calibration varies per call site, the functor accepts the calibration data as an additional argument forwarded through `point_for()`:

```

struct CameraCalibration { double R[3][3]; double t[3]; };

template<>
inline constexpr auto frame_projection<world_frame, camera_frame> =
    [] (quantity_point_of<isq::length> auto qp, const CameraCalibration& cal) {
        return camera_frame + forward_transform(cal, qp.delta_from(world_frame));
    };

// At runtime:
CameraCalibration cal = load_calibration("cam.json");
auto cam_pt = world_pt.point_for(camera_frame, cal);    // cal forwarded to the
    projection functor

```

The calibration is passed at the call site and forwarded by `point_for()` to the `frame_projection` specialization. Different call sites can supply different calibrations without any synchronization concern.



## § 8.10 Parameter-dependent conversion: geoid undulation

Altitude conversions between mean sea level (MSL) and height above ellipsoid (HAE) depend on the geoid undulation, which varies with geographic position — a runtime value that cannot be encoded as a compile-time `relative_point_origin`. The variadic `point_for()` passes the location to the projection functor at the call site:

```
template<>
inline constexpr auto frame_projection<mean_sea_level, height_above_ellipsoid> =
    [] (quantity_point_of<isq::altitude> auto msl, geo_latitude lat, geo_longitude
        lon) {
        quantity undulation = geoid_undulation_at(lat, lon);
        return height_above_ellipsoid + (msl.delta_from(mean_sea_level) -
            undulation);
    };

// Usage:
waypoint wpt = {"EPPR", {54.25_N, 18.67_E}, mean_sea_level + 16 * ft};
quantity hae = wpt.msl_alt.point_for(height_above_ellipsoid, wpt.pos.lat,
    wpt.pos.lon);
```

This maintains full compile-time type safety: the origins remain NTTP values, and the type system still prevents mixing quantities from incompatible frames. Only the transformation parameters are runtime values.

**Scope limitation:** This does not solve per-instance runtime origins (drone fleet scenario from [Limitations of relative\\_point\\_origin](#)). Even with `world_alt.point_for(drone_frame, drone_id)`, the result type is `quantity<point<altitude[m], drone_frame>>` — the same for all drones. The library cannot prevent mixing positions from different drone instances at compile time.

## § 8.11 Projection requirements and failure modes

`frame_projection` specializations are user-provided callables. The library imposes the following requirements and specifies the following behaviors for edge cases:

**Out-of-bounds results.** If the target origin carries a bounds policy NTTP, `point_for()` applies that policy to the projected value after the callable returns — exactly as for direct construction at that origin. The callable itself is not required to range-check its output; the origin’s policy is the single enforcement point.

**Round-trip correctness.** A projection should be invertible so that round-tripping a value through  $A \rightarrow B \rightarrow A$  recovers the original. The library does not verify this automatically (doing so would require calling both directions and comparing, which is not feasible in general). Non-invertible projections (e.g., a lossy rounding projection) are not ill-formed, but they produce values where `point_for()` followed by `point_for()` in the reverse direction does not recover the original value. Such projections are strongly discouraged; this is explicitly unspecified behavior left to the user.

**Composition consistency.** The library does not check that composing projections  $A \rightarrow B$  followed by  $B \rightarrow C$  agrees with a direct  $A \rightarrow C$  projection. Consistency is a user responsibility. Where a direct  $A \rightarrow C$  projection exists, the library uses it; where it does not, it chains  $A \rightarrow B$  and  $B \rightarrow C$ . If these disagree, results are numerically different but not ill-formed.

**NaN and infinity.** The library does not intercept NaN or infinity inputs to a projection callable. If the callable propagates NaN or infinity to the result, the behavior follows the representation type’s arithmetic rules. For floating-point representations, this is well-defined (IEEE 754); for other representations, it is implementation-defined. The target origin’s bounds policy applies to the (possibly NaN/infinite) result, but `check_in_range` and `check_non_negative` with NaN input render the result unspecified — the comparison `NaN < bound` is false by IEEE 754, so the check does not fire. Users who need NaN-safe projection should apply a guard inside the callable.

**noexcept.** Projection callables are not required to be `noexcept`. If a callable throws, the exception propagates through `point_for()` to the caller. A projection that is declared `noexcept` and throws terminates the program via the standard `noexcept` rules.

**Constant evaluation.** If a `frame_projection` specialization is `constexpr`-friendly (the callable itself is `constexpr` and its body is valid in a constant expression), `point_for()` is usable during constant evaluation. If the callable produces a value outside the target origin’s bounds, the target’s bounds policy applies under the same contract-mode rules as for direct construction. A specialization that is not `constexpr`-friendly makes the enclosing `point_for()` call non-constant — not an error in a non-`constexpr` context, but it prevents use in a `constexpr` initializer or template argument.

## § 8.12 Scope limitations

`frame_projection` is designed for **scalar, static-geometry frame relationships**: altitude ↔ depth, NED ↔ ENU convention flips, static sensor mount offsets, and singleton runtime calibrations. Multi-dimensional kinematic chains (attitude estimation, robot forward kinematics) are explicitly out of scope — they belong in a linear algebra library that uses quantity types as leaf elements.

# § 9 Text output for quantity points

## § 9.1 Current state

The library provides `std::formatter` specializations for quantity but not for `quantity_point`. The rationale is that a point’s origin is context-dependent and the library cannot determine how to format it. A point expressed as `42 m` could mean 42 meters above sea level, 42 meters above ground level, or 42 meters from the center of Mars.

## § 9.2 The challenge of default representations

Unlike units (meters, kilograms) or physical constants (speed of light), quantity points have no standard symbol or notation. Some domains use **postfix conventions** (“AMSL” for altitude above mean sea level), others use **prefix conventions** (“BRG 045°” for bearing, “HDG 270°” for heading), and many use no special notation at all.

The fundamental issue is illustrated by a simple example. Consider an air conditioning controller with a reference temperature of 21 °C. A user sets the temperature to 19 °C — that is, 2 °C below the reference:

```
inline constexpr quantity ac_reference = reference_temp + 21 * deg_C; // 294.15 K
quantity set_temp = ac_reference - 2 * deg_C; // 292.15 K
(19 °C)
```

This same physical point (292.15 K) can be represented from multiple origins:

```
quantity temp1 = ac_reference - 2 * deg_C;           // -2 °C from AC reference
quantity temp2 = ice_point + 19 * deg_C;             // +19 °C from ice point (0 °C)
quantity temp3 = absolute_zero + 292.15 * K;         // +292.15 K from absolute zero

assert(temp1 == temp2 && temp2 == temp3);           // All represent the same point
```

All three compare equal — they are the identical physical state. **What should the library print?**

- "-2 °C" (relative to AC reference)?
- "19 °C" (relative to ice point)?
- "292.15 K" (relative to absolute zero)?

All three representations are mathematically valid. Defaulting to any single choice risks confusion. Printing different text for equal values violates the principle of substitutability.

The same issue arises with timestamps. A quantity point representing 2024-03-15T14:30:00Z is the same instant regardless of whether it is stored relative to the Unix epoch (1970), the GPS epoch (1980), or an application-specific time origin. Users expect "2024-03-15T14:30:00Z" as output — but a units library cannot produce this. Formatting a timestamp as a calendar date requires knowledge of leap seconds, time zones, and calendar arithmetic — domain-specific logic that belongs in `std::chrono`, not in a units library.

## § 9.3 Resolution through absolute quantities

Absolute quantities resolve the most common case: when the origin is the natural zero of the physical quantity (0 kg, 0 m, 0 K), the displacement *is* the absolute value and its textual representation is unambiguous:

```
quantity mass = 42 * kg;
std::println("{} ", mass);           // "42 kg" — absolute quantity, clear meaning
```

The vast majority of quantities that users previously modeled as points *solely to express non-negativity or absolute amounts* (total mass, cumulative distance, etc.) will now be modeled as absolute quantities with full formatting support.

## § 9.4 Recommendation for general points

For points with non-trivial origins, **text output may be better left to domain-specific libraries and user customization:**

- `std::chrono` handles timestamps with calendar awareness
- Geographic libraries format coordinates with hemisphere indicators
- Engineering applications format altitudes with domain-appropriate conventions (AMSL, AGL, etc.)

The generic units library provides the underlying quantity point abstraction and enables users to extract and format the displacement from any origin:

```
quantity alt = mean_sea_level + 1350 * m;
std::println("{} AMSL", alt.delta_from(mean_sea_level)); // "1350 m AMSL"
```

Imposing a single default representation for points that can be expressed multiple ways may do more harm than good. The design space for general point formatting remains open for future work.

## § 10 Integer division safety

### § 10.1 Background

As described in [Integer division hazard](#), dividing two integer-valued quantities whose units differ performs raw integer division on the stored values, with units composed without prior normalization. The result is well-defined C++ arithmetic, but may be potentially surprising to users who expect unit normalization before division occurs.

Division by a raw integer, or by a quantity with the same unit, is not affected: in both cases the raw-value division produces the expected result:

```
quantity half = (8 * h) / 2;           // 4 h – safe (scalar denominator)
quantity ratio = (8 * h) / (2 * h);    // 4 – safe (same unit, no conversion
                                     factor)
```

### § 10.2 The `operator%` asymmetry

Addition, subtraction, and modulo are **common-unit operations**: both operands must share a dimension, and they are converted to their common unit before the operation is applied. Division, by contrast, is an **arbitrary-unit operation**: it applies to any pair of quantities — including different dimensions — and simply composes their units without converting them.

For `operator%`, common-unit semantics are the only meaningful choice. “How much remains after removing whole multiples of the denominator?” is naturally answered in the smallest unit of the two operands. As Richard Smith observed during review of [\[P3045R7\]](#): “*for homogeneous operators like + or %, it seems like the only reasonable option is that you get the `std::common_type` of the units of the operands.*”

```
static_assert(5 * h % (120 * min) == 60 * min); // 300 min % 120 min → 60 min ✓
static_assert(61 * min % (1 * h) == 1 * min);   // 61 min % 60 min → 1 min ✓
```

Division must use arbitrary-unit semantics to remain consistent across all uses, including cross-dimension division ( $\text{km} / \text{h} \rightarrow \text{km/h}$ ) where there is no common unit. Silently converting to a common unit for the same-dimension case would be inconsistent with the general case, and would risk overflow when the common unit is smaller than either operand.

As a result, for integer-valued quantities with different same-dimension units, `/` and `%` belong to different operation families and are not paired by the quotient-remainder theorem:

```
quantity q = (5 * h) / (120 * min); // arbitrary-unit: 5 / 120 = 0 (dimension-
                                     less h/min)
quantity r = (5 * h) % (120 * min); // common-unit: 300 % 120 = 60 min
// q * (120 * min) + r ≠ 5 * h      // quotient-remainder theorem does not apply
                                     here
```

This asymmetry is intentional and correct: `/` and `%` serve different roles for quantities. The `divide_in_common_unit` function proposed below provides the common-unit counterpart to `/` for the same-dimension case, restoring the quotient-remainder relationship.

## § 10.3 Current direction

The current [P3045R7] design, as agreed with SG6, permits same-kind cross-unit integer division without restriction: the raw stored values are divided directly and the units are composed. No utility functions for safe division were proposed or discussed. This is the approach documented in the [Integer division hazard](#) motivation section.

## § 10.4 New information from the Au library

Since that discussion, the Au library [Au] — which has extensive real-world experience with integer-backed quantities — has developed and deployed a stricter approach that was not presented to SG6 at the time. After years of iteration, the Au team found that silently allowing same-kind different-unit integer division caused enough user confusion that they chose to **block it at compile time** by default, and to provide two utility functions as explicit alternatives:

- **divide\_in\_common\_unit(a, b)** — converts both operands to their common unit before dividing, restoring the quotient-remainder relationship with `operator%`:

```
quantity ratio = divide_in_common_unit(8 * h, 40 * min);  
// 480 / 40 = 12 ✓  
  
// quotient-remainder theorem now holds:  
quantity q = divide_in_common_unit(5 * h, 120 * min); // 2  
quantity r = (5 * h) % (120 * min); // 60 min  
// q * (120 * min) + r == 5 * h // ✓
```

This only applies when both quantities have the same quantity kind (there is no “common unit” for length / time).

- **unblock\_int\_div(denominator)** — an explicit opt-in that suppresses the guard when the user has verified that raw-value division is intentional. It composes naturally with generic programming: the wrapper can be applied at the call site without changing the function signature:

```
quantity ratio = 8 * h / unblock_int_div(40 * min); // user takes  
responsibility
```

The question for SG6 is whether to revisit the earlier direction and adopt this stricter approach. The key new observation, reported by Chip Hogg, is that even experienced users of the Au library have been found reaching for `unblock_int_div()` as a quick way to make the code compile — without realizing that the blocked case was one where `divide_in_common_unit()` would have been the correct tool. In those cases the user intended normalized division (expecting a result like `480 / 40 = 12`) but silenced the guard and obtained truncated raw-value division instead (getting `8 / 40 = 0`), with no diagnostic. This pattern suggests that the error message must prominently direct users toward `divide_in_common_unit()` as the first choice, with `unblock_int_div()` reserved for the cases where raw-value division is genuinely intended.

Concretely, the stricter approach would be:

1. **Consider blocking same-kind cross-unit integer divisions at compile time.** When both operands of `quantity / quantity` have integral representation types, belong to the **same quantity kind**, and the denominator’s unit is not unit-equivalent to the numerator’s unit (i.e., the conversion factor between them is not exactly 1), the `operator/` overload is removed

from the overload set. Division of quantities of **different kinds** (e.g., km / h producing km/h), division by a raw scalar, and division by a quantity with the same unit are not affected:

```
quantity speed = 100 * km / (2 * h); // ✓ different kinds: length / time
quantity half  = (8 * h) / 2;        // ✓ scalar denominator
quantity ratio = (8 * h) / (2 * h);  // ✓ same unit, conversion factor = 1
auto bad      = (8 * h) / (40 * min); // ✗ ill-formed: same kind, different
                                     units
```

2. Consider providing `divide_in_common_unit(a, b)` (as above) as the recommended replacement for the common case.
3. Consider providing `unblock_int_div(q)` (as above) as an explicit opt-in for the cases where raw-value division is intentional.

## § 10.5 Scope

The constraint would apply only to `quantity / quantity` with integral representations where both operands belong to the **same quantity kind** and their units differ by a non-trivial conversion factor. Division of quantities of **different kinds** (e.g., distance / time producing a speed) is not affected — that case has no common unit to normalize to. Floating-point division is also unaffected (no silent truncation to zero). Division by a raw integer or by a quantity with the same unit is always allowed.

Using SFINAE rather than `static_assert` means the overload participates in overload resolution only when the division is well-formed, allowing `unblock_int_div` to be easily used as an alternative otherwise.

The author's preference is to **retain the current [P3045R7] direction** — permitting same-kind cross-unit integer division — while making `divide_in_common_unit` available as explicit opt-in for users who need them. The Au library evidence is real, but the primary concern with the stricter approach is **generic programming**: `quantity`, `double`, and other numeric types should be substitutable in generic algorithms. Blocking `quantity / quantity` for same-kind integer operands creates a divergence from `double / double`, which always compiles — a generic template that divides two values of the same type would silently break when `double` is replaced by an integer-backed quantity. The preferred remedy is therefore documentation and tooling guidance directing users toward `divide_in_common_unit`, not a restriction that undermines substitutability with other numeric types.

Straw poll: *Same-kind cross-unit integer division should be permitted (status quo, as in [P3045R7]) rather than blocked at compile time with `unblock_int_div` as the explicit opt-out.*

## § 11 Comparison against zero

### § 11.1 The problem

Zero is special. It is the only constant that unambiguously specifies the value of any quantity regardless of its units: zero inches and zero meters and zero miles are all identical. For this reason, it is very common to compare a quantity against zero — when checking its sign, asserting it is nonzero, or guarding a division.

The naive implementation requires repeating the units in the comparand:

```
if (q1 / q2 != 0 * m / s)
    // ...
```

This compiles, but is unsatisfying: if `q1 / q2` happens not to be in `m/s`, a superfluous unit conversion is incurred. Even when the units match, spelling them out is boilerplate. Hoisting the result into a temporary removes the conversion:

```
if (auto q = q1 / q2; q != q.zero())
    // ...
```

but this is cumbersome, and novice users may be unaware of the `.zero()` member or its rationale.

Three distinct strategies have been explored across the revision history of [\[P3045R6\]](#) and [\[P3045R7\]](#) for making comparisons against zero ergonomic without sacrificing safety.

## § 11.2 Strategy 1: Named comparison functions (`is_eq_zero` etc.)

[\[P3045R6\]](#) proposes a family of named comparison functions, following the naming convention of the C++ standard's [named comparison functions](#):

- `is_eq_zero(q)`
- `is_neq_zero(q)`
- `is_lt_zero(q)`
- `is_gt_zero(q)`
- `is_lteq_zero(q)`
- `is_gteq_zero(q)`

These call `.zero()` internally and are applicable to any type exposing that member — not only quantities, but also `std::chrono::duration` and any other user-defined type:

```
if (is_neq_zero(q1 / q2))
    // ...
```

### Advantages:

- No operator overloading or overload resolution surprises.
- Generic: works uniformly across every compatible type.
- Explicit: the intent is immediately readable from the function name.

### Disadvantages:

- Introduces six new API names to learn.
- Replaces the familiar operator syntax (`q > 0`) with a function call (`is_gt_zero(q)`), making code harder to read and diffs harder to review.
- Makes it harder to write generic code that works for both `double` and `quantity`: a template that uses `is_gt_zero(x)` does not compile when `x` is a plain `double`, so callers cannot drop in a raw scalar without a wrapper.



- Makes it harder to migrate existing `double`-based code to quantities: every comparison against `0` must be mechanically rewritten as a named-function call, rather than just changing the type of the variable.

## § 11.3 Strategy 2: A Zero tag type

The `[Au]` library takes a different approach: an empty type `Zero` whose sole purpose is to represent the compile-time numeric zero. Every quantity is implicitly constructible from `Zero`, and a global constant `ZERO` is provided for use at call sites.

```
if (q1 / q2 > ZERO)
    // ...
```

This preserves the operator syntax and requires no new function names. It is particularly natural when migrating existing code from raw numeric types:

```
// Before: if (speed_sq > 0)    { ... }
// After:  if (speed_sq > ZERO) { ... }
```

### Advantages:

- Preserves the familiar operator-based style.
- Requires learning only a single new name that covers all six comparison forms.
- Easy to explain: “use `ZERO` wherever you would write `0` in a quantity comparison.”

### Disadvantages:

- `Zero` is not itself a quantity, which causes failures in generic interfaces:

```
namespace v1 { void foo(quantity<si::metre> q); }
namespace v2 { void foo(QuantityOf<isq::length> auto q); }

v1::foo(ZERO);    // OK
v2::foo(ZERO);    // Compile-time error
```

- Is not associated with any unit so it cannot be used with point origins that need both a unit and a value:

```
msl_altitude alt = mean_sea_level + 0 * si::metre; // OK
msl_altitude alt = mean_sea_level + ZERO;           // Compile-time error
```

- Does not extend to multiplication (`q * ZERO` has no known result type or unit) — `ZERO` is not a scalar zero.
- Makes generic code that works for both `double` and quantity harder to write: a template comparing `x > ZERO` does not compile when `x` is a plain `double` (unless `Zero` is made implicitly convertible to `int{0}`, which would address comparison but still not multiplication).
- Makes migration from `double`-based code more involved: `> 0` becomes `> ZERO` rather than being left unchanged.
- Novices may over-apply `ZERO` and become confused when it fails in non-obvious cases.

## § 11.4 Strategy 3: Literal `0` with compile-time guard

[P3045R7] adopts a third approach: the six comparison operators are overloaded — as hidden friends — to accept a compile-time constant that is exactly `0`, using a `constexpr`-based check to reject any other value:

```
if (q1 / q2 != 0)
    // ...
```

Only a compile-time literal that evaluates to zero is accepted: `0`, `0.`, `0.f`, `0LL`, etc. A runtime variable, or a non-zero literal, is rejected at compile time.

Internally, this strategy uses Strategy 2's zero tag type under the hood: the hidden-friend operator accepts a zero parameter, and zero has a `constexpr` constructor that accepts any scalar value but rejects non-zero values at compile time. This is how writing `q > 0` in user code triggers the guard — the literal `0` implicitly constructs zero via the `constexpr` constructor, and any non-zero literal or runtime value causes a compile error at that constructor. A simplified sketch of the implementation in [mp-units]:

```
struct zero {
    template<typename T>
    constexpr zero(T v) { if (v != T{}) throw "only literal zero is accepted"; }
};

// Hidden friend inside quantity<R, Rep>:
[[nodiscard]] friend constexpr bool operator==(const quantity& lhs, zero)
{
    return lhs.numerical_value_ref_in(get_unit(R)) ==
           representation_values<Rep>::zero();
}
```

#### Advantages:

- Preserves natural operator syntax with no new user-facing names.
- Covers all six comparison operators without additional API surface.
- Hidden friendship avoids polluting the global namespace and prevents most unexpected overload resolution interactions with third-party types.
- Works identically for plain arithmetic types (`double`, `int`, ...): a generic template that compares `x > 0` requires no change when `x` is a quantity instead of a `double`.
- Zero migration cost: existing `double`-based code that compares against `0` compiles unchanged after the variable type is changed to quantity.

#### Disadvantages:

- Comparing without explicit units is, from a metrological standpoint, physically questionable: zero *what*? Peter Hrenka raised this concern during Croydon 2026 review: “*We are trying to train people to use units, but not in this case.*” Oliver Rosten similarly objected that it allows comparing oranges and apples.
- The `constexpr` mechanism was briefly removed between revisions because some compiler implementations did not support the required constructor semantics at that time.

## § 11.5 Discussion and current status

All three strategies make different trade-offs. Strategy 1 (named functions) is the most explicit and metrologically safe, but imposes a naming tax, disrupts the familiar operator-based style, and breaks generic code that must also work with plain `double`. Strategy 2 (Zero tag type) restores the operator style and requires learning only one new name, but silently fails in concept-constrained generic interfaces and still requires a mechanical change at every migration site (`0`  $\rightarrow$  `ZERO`). Strategy 3 (literal `0`) preserves ordinary syntax with no new names, is fully compatible with generic `double`/quantity templates, and has zero migration cost — but makes a metrologically questionable exception to the general rule that every quantity comparison must specify a unit.

The author’s current recommendation is **Strategy 3** (literal `0` with compile-time guard). The unit-correctness objection is noted but the practical cost is low: zero is the one constant that is genuinely unit-independent, and restricting the exception to exactly the literal `0` — not a variable, not a non-zero constant — keeps the special case narrow and auditable. The generic-code compatibility and zero migration cost are decisive advantages over Strategies 1 and 2 for a library whose primary users are domain experts porting existing `double`-based physics code.

Straw poll: *Comparison against literal `0` should be permitted via hidden-friend operator overloads (Strategy 3) as the standard zero-comparison mechanism for quantity.*

## § 12 Summary

The following table summarizes the proposed additions and their relationship to [\[P3045R7\]](#):

| Feature                   | Impact on <a href="#">[P3045R7]</a>                    | Release | Status                                                                  |
|---------------------------|--------------------------------------------------------|---------|-------------------------------------------------------------------------|
| Non-negative quantities   | Pure addition (ISQ spec definitions)                   | First   | Implemented in <a href="#">[mp-units]</a>                               |
| Absolute quantities       | Breaking change (default)                              | First   | Not in <a href="#">[mp-units]</a> ; proven in <a href="#">[Sequoia]</a> |
| Affine space annotations  | Breaking change (unifies <code>quantity_point</code> ) | First   | Not yet implemented                                                     |
| Range-validated points    | Pure addition                                          | First   | Implemented in <a href="#">[mp-units]</a>                               |
| Runtime frame projections | Pure addition                                          | Later   | Not yet implemented                                                     |
| Text output for points    | Resolved by absolutes                                  | First   | Indirect resolution                                                     |
| Integer division safety   | Breaking change (guard)                                | First   | Implemented - open for SG6 decision                                     |
| Comparison against zero   | Design choice (open)                                   | First   | Implemented - open for SG6 decision                                     |

The **Release** column classifies each feature as:

- **First**: must be part of the initial [\[P3045R7\]](#) release. These features either involve breaking changes that cannot be introduced later without an ABI or API break, must be included in the initial ISQ quantity spec definitions (after which adding them would require reopening those definitions), are required to implement the non-negativity enforcement that `non_negative` quantity specs imply (range-validated points power the automatic `check_non_negative` policy

on `natural_point_origin`), or resolve problems (text output, integer division, zero comparison) whose solutions must be established before the initial API is locked.

- **Later:** pure additions that can be shipped in a follow-on proposal without affecting existing code. They build on the first-release foundation but do not constrain it.

Three changes affect the initial [P3045R7] design as breaking changes: absolute quantities (which change the default meaning of `quantity<R>` from delta to absolute), the affine space annotations (which unify `quantity_point` into `quantity<point<R>>`), and the integer division safety guard (which, if adopted by SG6, removes `operator/` for same-kind cross-unit integer division). All other first-release features are pure additions, but must ship together because they are interdependent: `non_negative` quantity specs require range-validated points to implement automatic enforcement on `natural_point_origin`, and both depend on absolute quantities providing the correct value abstraction. Only runtime frame projections can be introduced independently in a later standard.

## § 13 Next steps

If SG6 gives a favorable direction on the core abstractions (absolute quantities and the three-way delta/absolute/point split), the path forward is:

1. **Implement the two unimplemented breaking changes** — absolute quantities and affine space annotations — in the [mp-units] reference implementation. Actual implementation experience may yield minor refinements to the arithmetic semantics and hierarchy rules presented in this paper.
2. **Validate against real-world use cases.** The existing [mp-units] user base will exercise the new abstractions against production physics, embedded systems, and financial code. Any design issues surface at this stage before standardization.
3. **Merge the approved extensions into [P3045R7].** The extensions proposed here are designed as additions to and breaking changes within [P3045R7]; they do not require a separate paper. A future revision of [P3045R7] will incorporate the agreed extensions and remove the design alternatives not chosen by SG6.
4. **Submit the merged paper to LEWG** for design review and approval.
5. **Submit to LWG** for full wording review once LEWG has approved the design.

Runtime frame projections (the “Later” feature in the summary table) follow a parallel track: they can be implemented and proposed independently once the core abstractions are stable, without blocking the initial standardization of [P3045R7].

## § 14 Acknowledgments

Special thanks and recognition goes to [The C++ Alliance](#) for supporting Mateusz’s membership in the ISO C++ Committee and the production of this proposal.

The author also thanks **Oliver Rosten** for independently developing the convex-space model [Rosten2025] that contributed the four-space taxonomy and several key design insights used throughout this paper; **Tiago Freire** for the ideal gas law demonstration that motivated the temperature trap analysis and for detailed ANSI review; **Chip Hogg** for reporting the integer division experience

from the [Au] library and suggesting the nature-based constant idiom; **Yongwei Wu** for raising the algebraic identity objection against absolute subtraction; **Peter Hrenka** for the zero-comparison unit-correctness concern raised during Croydon 2026; **Amir Kirsh** for review feedback; and **Richard Smith** for helping resolve the integer-modulo problem during early [P3045R7] review.

## § 15 References

[Au] The Au Units library.

<https://aurora-opensource.github.io/au>

[mp-units] mp-units - A Physical Quantities and Units library for C++.

<https://mpusz.github.io/mp-units>

[P3045R6] Mateusz Pusz, Dominik Berner, Johel Ernesto Guerrero Peña, Charles Hogg, Nicolas Holthaus, Roth Michaels, Vincent Reverdy. 2025-06-19. Quantities and units library.

<https://wg21.link/p3045r6>

[P3045R7] Mateusz Pusz, Dominik Berner, Johel Ernesto Guerrero Peña, Charles Hogg, Nicolas Holthaus, Roth Michaels, Vincent Reverdy. 2026-02-23. Quantities and units library.

<https://wg21.link/p3045r7>

[P3380R1] Barry Revzin. 2024-12-17. Extending support for class types as non-type template parameters.

<https://wg21.link/p3380r1>

[Rosten2025] Oliver J. Rosten. Thoughts on Quantities & Units — Putting P3045 on a firm foundation.

[https://docs.google.com/presentation/d/1qjw0SURoJtX\\_DhaFUr62F-BXtQJjdQRqzMUHvQ6h3-E/edit?usp=sharing](https://docs.google.com/presentation/d/1qjw0SURoJtX_DhaFUr62F-BXtQJjdQRqzMUHvQ6h3-E/edit?usp=sharing)

[Sequoia] Oliver J. Rosten. Sequoia — a C++ library exploring mathematical foundations for quantities.

<https://github.com/ojrosten/sequoia>

[Williams2025] Anthony Williams. Units and Anchors (2025-01-26).